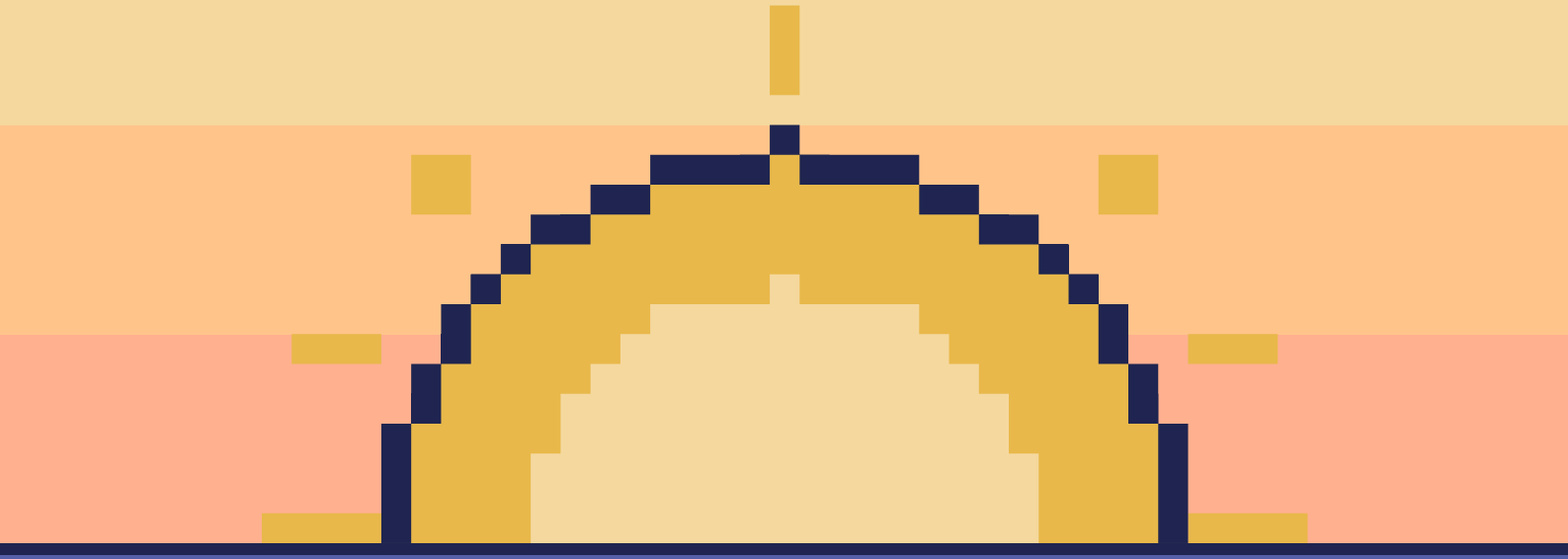
ARAW

Technical Documentation

V1.0

Architecture, content, games, languages, offline use, the Chatbot Sandbox, pipelines, security, testing and maintenance

PROJECT PARTNERS



AN AIKASALIG PROJECT · DOST-PCHRD · NATIONAL UNIVERSITY

AI Relevance Activities and Workshops · 3 November 2026

Contents

1 Overview 3

2 System architecture 5

3 Technology stack and third-party components 7

4 Repository layout 9

5 App shell, routing, state and settings 10

6 Content model 13

7 Learning modules and the lesson/card engine 16

8 Game engine and arcade games 18

9 Internationalisation 21

10 Accessibility implementation 23

11 Offline and PWA 24

12 Chatbot Sandbox 26

13 Media and document pipelines 30

14 Privacy and security 33

15 Build, quality gates and testing 35

16 Deployment and configuration 37

17 Maintenance runbooks 39

18 Known limitations and roadmap 42

A npm scripts and tool commands 44

B Glossary of terms 45

C Decision-log index 46

ARAW: AI Relevance Activities and Workshops · a project of AIKasalig, DOST-NICER R&D Center on Natural Language Processing for Health Applications, National University, Manila, with support from DOST-PCHRD.

Document	ARAW V1.0 Technical Documentation, 3 November 2026
Audience	Technical reviewers (DOST-PCHRD, NICER), maintainers and future developers
Application	`araw` package version 0.1.0 (shown in the app and in reports as "0.1.0 beta"); deliverables are labelled V1.0
Live app	https://dost-nicer-araw.mjmagtira.workers.dev/
Source	https://github.com/MicoMagtira/ARAW-AI-Relevance-Activities-and-Workshops (branch `main`)
Licences	Code MIT (`LICENSE`); content CC BY 4.0 (`CONTENT-LICENSE.md`)

Every statement in this document is based on the repository at the time of writing (60 commits on `main`). Numbers such as content counts and test counts were measured on that snapshot and are marked as such. Where the repository does not settle a question, the text says so (see section 18.1).

SECTION 1

Overview

1.1 Purpose

ARAW is a gamified, mobile-first and desktop-friendly web app that teaches **AI literacy** and **resilience to health misinformation** to non-technical Filipinos: seniors, parents, students and health workers. It is the hands-on AI literacy product of the AIKasalig project ("Advancing Responsible AI in Philippine Healthcare: A Multi-Year NLP-Driven Roadmap for Trust, Transparency, and Ethical AI").

The teaching idea is **calibrated trust**, not "trust less". A player who calls every true health post "fake" has learned nothing useful, so scoring rewards discernment (telling true from false), and every module ends with the same card: "When should I trust it?".

1.2 Audiences

Audience	How they use ARAW
Learners (seniors, parents, students, health workers)	Ten short modules, an arcade of eleven games, Fact or Fake, a daily challenge, the Library, the Chatbot Sandbox. English, Filipino, Cebuano, Ilokano and Hiligaynon.
Facilitators of community workshops	Printable kit (posters, cards, session guides, certificate), module lecture videos, and the matched Fact or Fake pre-test and post-test sheets (facilitator-only).
Reviewers (medical, cultural, language)	`docs/CONTENT_REVIEW.md` lists every content entry with its basis; all entries ship as `review_status: "unreviewed"`.
Maintainers and developers	This document, `CLAUDE.md`, `docs/SPEC.md`, `docs/CONTENT_GUIDE.md`, `docs/DESIGN.md`, `docs/DECISIONS-LOG.md`.

1.3 Scope

In scope: the static web app (installable PWA), its content, its five-language runtime packs, and the tooling that produces the user guide PDF, the print kit, the how-to videos, the module lecture videos and this document.

Out of scope in the current release: any server, account system, analytics, or cloud database; native apps (an offline Windows `.exe` and an Android App Bundle are planned, see section 18.2); medical review of content (pending).

1.4 Project and partners

The About screen (`src/screens/About.tsx`, strings under `about.*`) and `src/ui/PartnerLogos.tsx` present the project identity:

- **Project:** AIKasalig, Project 4 of the program "Research and Development Center on Natural Language Processing for Health Applications (RDC-NLPHA)", part of DOST's NICER (Niche Centers in the Regions for R&D), implemented by National University, Manila. Duration: February 2026 to January 2029 (36 months).
- **Sister projects named in the app:** HealthPH+ (multilingual public health surveillance), AdvocAid PH (geospatial AI for HIV public health surveillance), Bantay Lamok PH (mosquito density and outbreak prediction), TALA (a learning module on text and location analytics, also from AIKasalig).
- **Partner logos, in fixed order:** DOST, DOST-PCHRD, National University, NICER-DOST, NICER RDC-NLPHA, AIKasalig. They are built from a git-ignored `logos/` folder by `scripts/logos/build_logos.py` into `src/assets/logos/*.webp` (plus `logos.json` with sizes) and are always shown in full colour on white.

- **Credits shown in the app:** Project Leader, AIKasalig: Dr. Mideth B. Abisado. ARAW developer: Mico C. Magtira, Senior Data and NLP-Geospatial Scientist, DOST-NICER.

1.5 Version and status

Item	Value	Source
Package version	0.1.0	`package.json`
Version string in app and reports	`0.1.0 beta`	`vite.config.ts` (`__APP_VERSION__`)
Service worker cache id	`<version>-<8 hex of SHA-1 of the precache list>`	`scripts/pwa-plugin.ts`
Node version	22 (`.node-version`); README states Node 20+	repo
Content review status	all `unreviewed`	every content file

The in-app "beta" and "not medical advice" banners were removed on 2026-10-16 (the proposal already frames ARAW as educational); the version suffix remains in the version string.

SECTION 2

System architecture

2.1 Client-only design

ARAW is a **static single-page application**. There is no application server, no database, no user account and no analytics. The built folder `dist/` is served as plain files; every computation (lessons, games, scoring, translation, even the chatbot) runs in the learner's browser. Consequences:

- It can be hosted for free on any static host (Cloudflare, Hugging Face static Space, GitHub Pages) and scales with the host's CDN, not with ARAW code.
- Progress lives only in the browser (`localStorage`); clearing site data clears progress.
- After the first visit it works offline (service worker), except features that need the network by nature (the first model download, the optional report send, the how-to videos and print PDFs, which stream or download on demand).

2.2 Diagram

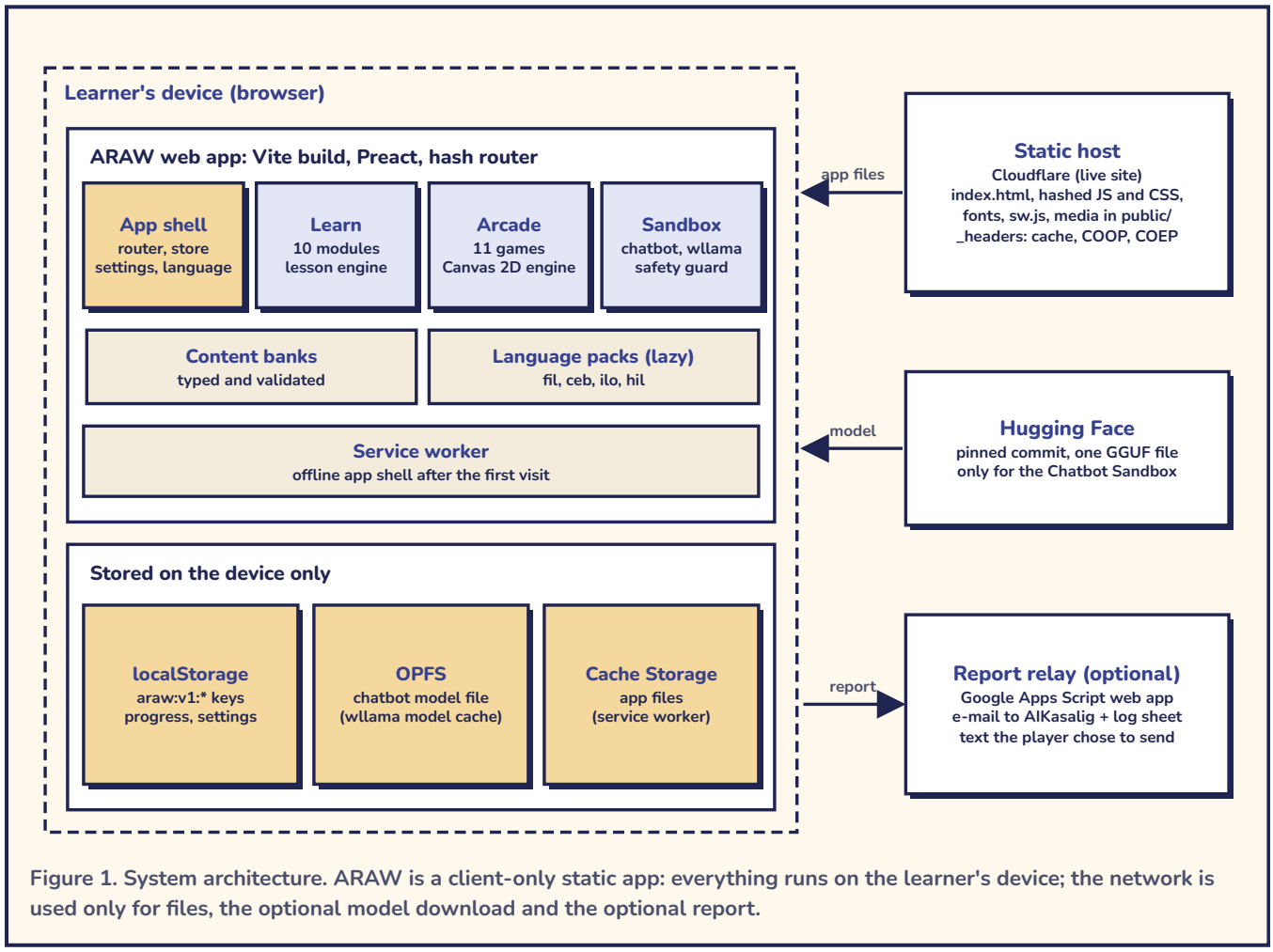


Figure 1. System architecture. ARAW is a client-only static app: everything runs on the learner's device; the network is used only for files, the optional model download and the optional report.

2.3 Boot and data flow

1. `index.html` loads the entry script `src/main.tsx`, which installs the translation hook, applies saved settings as root classes, starts the hash router and sound controller, then loads the saved language pack

before the first paint (English needs no pack).

2. `App` decides the boot step (`src/app/boot.ts`): on the first visit of a browser session that lands on Home, the language landing, then the sunrise splash, then the `Shell` . A deep link (for example a shared `#/learn/m03`) skips the intro. The "booted" flag is in `sessionStorage` (key `araw:v1:booted`) with an in-memory fallback.
3. `Shell` maps the current hash to a lazy screen (`src/app/routes.ts` , `router.ts` , `LazyScreen.tsx`). Each module's lesson data, each game and the Library are separate chunks that download on demand.
4. Screens read and write **signals** (`@preact/signals`). Persisted signals (`persisted()` in `src/app/store.ts`) validate data on read and write through to `localStorage` .
5. After the first visit the service worker serves the app shell and chunks from Cache Storage.
6. Only four kinds of requests leave the app: (a) same-origin files from the host; (b) caption files for videos (same origin); (c) the pinned model file from Hugging Face, only for the Chatbot Sandbox; (d) an optional POST of a player-written report to the relay, only when the player presses Send and a relay URL was configured at build time. The code contains no other `fetch` call and no `innerHTML` , `dangerouslySetInnerHTML` or `eval` .

2.4 Main building blocks

Block	Location	Role
App shell	`src/app/`	Router, layout, store, settings, language, progress, streak, daily challenge, sound, PWA registration
Shared UI	`src/ui/`	Pixel UI kit (`kit.tsx`), Ara sprite, icons, confetti, avatars, partner logos, tutorial player
Screens	`src/screens/`	Home, Learn, Lesson, Arcade, Library, Me, Daily, Report, Certificate, About, Guide, Sandbox
Modules	`src/modules/`	`m01` .. `m10` lesson content and module games, shared card renderers
Games	`src/games/`	Shared engine, 11 arcade games, Fact or Fake
Content	`src/content/`	Typed, validated banks and manifests
Sandbox	`src/sandbox/`	In-browser chatbot (wllama), safety guard, missions, preload
i18n	`src/i18n/` and `src/app/i18n.ts`	English catalog, runtime packs, DOM and canvas translation hooks
Tooling	`scripts/` , `tools/`	PWA plugin, budget check, i18n pipeline, media pipelines, report relay

SECTION 3

Technology stack and third-party components

3.1 Stack

Layer	Choice	Notes
Language	TypeScript 7.0.2, `strict`, `noUncheckedIndexedAccess`, `noUnusedLocals/Parameters`	`tsconfig.json`; `npm run build` type-checks first
UI framework	Preact 11.0.1 with `@preact/signals` 2.11.3	JSX via `jsxImportSource: preact`
Bundler	Vite 8.3.4 with `@preact/preset-vite`	`base: './'` (relative paths), target `es2022`, no source maps
Router	Hand-written hash router	static hosting, no server rewrites
Games	HTML Canvas 2D on a small shared engine (`src/games/engine/`), plus DOM games	no game framework
Sound	Web Audio synthesis (`src/app/sound/synth.ts`)	chiptune music and effects are generated live: no audio files
Unit tests	Vitest 5.0.4, jsdom 30, `@testing-library/preact`	1,101 tests in 68 files
End-to-end tests	Playwright 1.64.1 (Chromium), `@axe-core/playwright` 4.13.0	900 tests (300 per project) in 37 spec files
Persistence	`localStorage` (and `sessionStorage`, OPFS for the model)	always in try/catch
Size	about 38,400 lines of TypeScript and TSX, 11,200 lines of CSS in `src/`	measured

3.2 Runtime dependencies (shipped to learners)

Package	Version	Licence	Use
`preact`	11.0.1	MIT	UI
`@preact/signals`	2.11.3	MIT	state
`@wllama/wllama`	3.8.1	MIT	llama.cpp compiled to WebAssembly: Chatbot Sandbox engine
`@wllama/wllama-compat`	3.8.1	MIT	compatibility build for browsers without JSPI or 64-bit WebAssembly memory (mostly Safari)
`pixelarticons`	2.4.2	MIT	pixel-art icons (plus hand-made pixel SVGs)
`@fontsource/baloo-2`	5.3.0	OFL-1.1	headings
`@fontsource/nunito`	5.3.0	OFL-1.1	body text
`@fontsource/press-start-2p`	5.3.0	OFL-1.1	titles, buttons, HUDs, labels

Fonts are self-hosted from `node_modules` into the build; there are no CDNs at runtime. The About screen lists the licences to players.

3.3 Development and tooling dependencies (not shipped)

Package	Version	Licence	Use
`vite`, `vitest`	8.3.4, 5.0.4	MIT	build, unit tests
`typescript`	7.0.2	Apache-2.0	type checking
`@playwright/test`	1.64.1	Apache-2.0	e2e tests, and headless Chromium for PDFs, videos, screenshots
`@axe-core/playwright`	4.13.0	MPL-2.0	accessibility checks
`@testing-library/preact`, `@testing-library/dom`	3.2.4, 10.4.2	MIT	component tests
`jsdom`	30.1.3	MIT	unit-test DOM
`qrcode` (+ `@types/qrcode`)	1.5.4	MIT	QR codes on printed sheets
`ffmpeg-static`	5.3.0	GPL-3.0-or-later (package metadata)	encodes the module lecture MP4s; used only on a developer machine, never bundled into the app
`marked`	18.1.3	MIT	renders this document to HTML for the PDF
`@types/node`	26.6.5	MIT	typings

3.4 AI model and its licence

The Chatbot Sandbox loads **one small open model** at runtime from a pinned Hugging Face commit. The default is **Qwen3 0.6B**, Q4_K_M quantisation, 378 MB, licence **Apache-2.0**, with the model's "thinking" mode turned off (`src/sandbox/model.ts`). Five candidates were compared in a bake-off (decision log, 2026-10-21, SANDBOX-MODEL):

Id	Label	Size (MB)	Licence
`qwen3-0.6b` (default)	Qwen3 0.6B	378	Apache-2.0
`qwen3.5-0.8b`	Qwen3.5 0.8B	508	Apache-2.0
`qwen2.5-0.5b`	Qwen2.5 0.5B	409	Apache-2.0
`gemma3-270m`	Gemma 3 270M	241	Gemma Terms of Use
`gemma3-1b`	Gemma 3 1B	688	Gemma Terms of Use

Only the default is used by learners. The Gemma candidates are test options (`#/sandbox?model=gemma3-1b`); the Gemma Terms of Use would need review before any of them became the default. Model weights are not stored in the repository.

3.5 Content and asset licences

Asset	Licence
Source code	MIT
Lessons, items, myths, glossary, Comics, art	CC BY 4.0, credited to AIKasalig, National University and DOST
Third-party icons and fonts	MIT and OFL as above
Partner logos	belong to the respective organisations; used to present the project identity

SECTION 4

Repository layout

```
app/
  CLAUDE.md, README.md, LICENSE, CONTENT-LICENSE.md      developer brief, overview, licences
  ARAW V1.x-User-Guide.pdf                                printable user guide (root copy; the name carries its
version)
  ARAW V1.0-Technical-Documentation.pdf                   this document as PDF
  index.html, vite.config.ts, playwright.config.ts, tsconfig.json, package.json, .node-version
  public/                    copied as-is into dist/
    _headers                 caching + security headers (Cloudflare format)
    manifest.webmanifest, favicon.svg, icons/        PWA manifest and icons
    videos/<id>/<lang>.webm|vtt                      how-to videos and captions (13 clips x 5 languages)
    print/<lang>/<id>.pdf                            public print kit (10 sheets x 5 languages)
    guide/                                           user-guide screenshots (WebP) and ARAW-User-Guide.pdf
  src/
    main.tsx                 entry
    app/                     shell, router, store, settings, language, progress, streak, daily, sound/
    ui/                      shared components, Ara, icons, komiks scenes, tutorial player
    screens/                 Home, Learn, Lesson, Arcade, Library/, Daily/, Sandbox/, Guide/, Certificate/ ...
    modules/                 m01..m10, cards/, engine.ts, registry.ts, types.ts
    games/                   engine/, fact-or-fake/, one folder per arcade game, registry.ts
    content/                 typed banks, factOrFake/, library/, manifests (JSON)
    sandbox/                 chatbot engine, guard, prompt, missions, preload, wllama backends
    i18n/                    runtime.ts, dom.ts, packs/{fil,ceb,ilo,hil}.json
    styles/                  tokens.css, base.css and one stylesheet per area
    assets/logos/            partner logos (WebP)
  scripts/
    pwa-plugin.ts, sw-template.js        service worker generation
    check-budget.mjs                      performance budget
    make-icons.mjs                        PWA icons from a 16x16 pixel mark
    i18n/                                 extract, batches, pack, review + README, TRANSLATOR, glossaries
    tutorials/                            how-to video recorder and storyboards
    lectures/                             module lecture video recorder
    guide/                                user-guide capture and PDF
    print/                                print kit (a small Vite + Preact app, prints with Playwright)
    sandbox/                              model fetch and real-model smoke tests
    audit/, shots/                        layout audit and release screenshot sweeps
    techdoc/                              this document's PDF builder
    logos/                                logo builder (Python)
  tools/report-relay/                     Google Apps Script relay (Code.gs, README)
  tests/unit/  tests/e2e/                 Vitest and Playwright
  docs/                                   SPEC, DESIGN, CONTENT_GUIDE, CONTENT_REVIEW, ARCHITECTURE,
                                          DECISIONS-LOG, RELEASE-CHECKLIST, PRINT-KIT-PLAN, TECHNICAL
```

Git-ignored outputs: `node_modules/`, `dist/`, `shots-out/`, `test-results/`, `logos/` (source artwork), `public/models/`, `scripts/print/out/`, `print-out/` (facilitator-only test sheets), `dist-lecture/`, `lecture-out/`, `scripts/techdoc/out/`.

SECTION 5

App shell, routing, state and settings

5.1 Shell and responsive layout

`src/app/Shell.tsx` picks the layout by viewport:

Width	Navigation	Header
1024 px and up (primary design)	left sidebar (logo, Home, Learn, Arcade, Library, Me, report link, credit)	gold banner with screen title, XP and streak chips, avatar
600 to 1023 px	sticky top tab bar with the same five items	gold header
under 600 px	fixed bottom tab bar	compact gold header

Focus mode: the routes `lesson`, `fof`, `assess`, `game` and `certificate` render without sidebar, banner or tab bar (their own top bar), so the player is not distracted. A "Skip to content" link is provided.

Layouts are fluid (flex, grid, `clamp()`, container queries) with no horizontal page scroll from 320 px to 2560 px; every e2e spec asserts this (section 15).

5.2 Route table

The router parses `location.hash` (`#/path?query`) and matches `src/app/routes.ts` in order. Unknown paths give the Not Found screen.

Pattern	Route name	Screen	Tab highlighted
`/`	home	Home (path, continue card, daily, project banner)	Home
`/learn`	learn	Learn path of the 10 modules	Learn
`/learn/:id`	lesson	Lesson runner for module `m01` .. `m10` (focus mode)	Learn
`/play/fact-or-fake`	fof	Fact or Fake free play (focus mode)	Arcade
`/assess/:form`	assess	Hidden Fact or Fake test forms `form_A` or `form_B`; never linked in the UI	none
`/arcade`	arcade	Arcade list and details	Arcade
`/arcade/:slug`	game	One arcade game, canvas or DOM (focus mode); `demo` is a test-only slug	Arcade
`/daily`	daily	Daily Challenge	Arcade
`/sandbox`	sandbox	Chatbot Sandbox	Arcade
`/library`	library	Library, default tab Myths	Library
`/library/:tab`	library	`myths`, `glossary`, `komiks`, `videos`, `printables`	Library
`/library/komiks/:n`	library	Comics reader for episode `n`	Library
`/me`	me	Profile, stats, settings, language, reset	Me
`/certificate`	certificate	Completion certificate (after all 10 modules; focus mode)	Me
`/report`	report	Report a problem (query: `screen`, `item`)	none

Pattern	Route name	Screen	Tab highlighted
`/about`	about	About the project	none
`/guide`	guide	In-app user guide	Me
`*`	notFound	Not Found	none

The language landing and the sunrise splash are boot steps, not routes.

5.3 State and persistence

State is held in Preact signals. `src/app/store.ts` provides `persisted(key, fallback, validate)`: a signal initialised from `localStorage` (through the validator, so corrupt or tampered values fall back to defaults) and written back by an effect. Keys are namespaced and versioned with the prefix `araw:v1:` so a future schema can live beside this one.

Key (after `araw:v1:`)	Type	Meaning	File
`profile`	`{nickname (max 20), avatar}`	player nickname and avatar index	`state.ts`
`xp`	number	total experience points	`state.ts`
`streak`	`{current, best, lastDay, freezeWeek}`	day streak, Asia/Manila dates, weekly freeze	`state.ts`, `streak.ts`
`modulesDone`	string[]	finished module ids	`progress.ts`
`cardsDone`	`{moduleId: n}`	lesson cards completed (resume point)	`progress.ts`
`moduleStars`	`{moduleId: 1..3}`	best module-game stars	`progress.ts`
`xpClaims`	string[]	XP already paid (`m01:card:m01-c1` , `m01:game` , `sandbox:ask` ...) so nothing pays twice	`progress.ts`
`badges`	string[]	earned badge ids (20 possible)	`progress.ts`
`bestRadar`	`{dPrime, c, total}` or null	best Truth Radar	`progress.ts`
`tacticStreaks`	`{villainId: n}`	Spot the Tactic streaks per villain	`progress.ts`
`villainCards`	string[]	collected villain cards	`progress.ts`
`komiksRead`	string[]	Comics episodes read	`progress.ts`
`bestScores`	`{slug: n}`	best arcade score per game	`scores.ts`
`briefingsSeen`	string[]	game briefings already seen	`scores.ts`
`daily`	`DailyRecord` or null	today's daily result (only the latest day is kept)	`dailyChallenge.ts`
`settings`	five booleans	accessibility settings	`settings.ts`
`language`	language code or null	chosen app language	`language.ts`
`sound`	`SoundPrefs`	music and effect switches and volumes	`sound/prefs.ts`
`tutorialCaptions`	boolean	captions on in video player (default true)	`tutorials.ts`
`booted` (`sessionStorage`)	`'1'`	splash already shown this session	`boot.ts`

The certificate name is typed per use and **not stored**. The chatbot's conversations live in memory only.

When storage is blocked or full: `createStorage()` wraps every access in try/catch. On failure it keeps values in an in-memory `Map` and reports `isPersistent() === false`; the app keeps working for that visit, but nothing is saved. `clearAll()` ("Reset my progress" on the Me screen, after a confirm dialog) removes only keys with the `araw:v1:` prefix. The same guard exists for `sessionStorage` in `boot.ts` and in the e2e fixtures.

5.4 Settings and accessibility classes

Settings are off by default and persisted under `settings`. `initSettings()` toggles one class on `<html>` per setting through an effect:

Setting	Root class
Larger text	`a11y-large-text` (sets `--text-scale`)
High contrast	`a11y-high-contrast`
Extra-large buttons	`a11y-xl-buttons`
Screen-reader hints	`a11y-sr-hints`
Reduce motion	`a11y-reduce-motion`

Components never hard-code colours: all styling uses CSS variables from `src/styles/tokens.css` (palette: cream, surface, gold, deep gold, navy, ink, muted, coral for fills only, fact green, fake red, line, soft). The 8-bit design rules (pixel borders, hard 4 px offset shadows, Press Start 2P for UI labels, Nunito 18 px body, step animations) are in `docs/DESIGN.md`.

SECTION 6

Content model

6.1 Principles

- English is the source language; the other four languages come from runtime packs (section 9).
- All people, pages, products, places and brands are **fictional**; phone numbers and IDs are masked placeholders; images are synthetic. No real names or likenesses.
- Health facts must be well-established (WHO/DOH/FDA Philippines level consensus). No dosing, no diagnosis, always point to health workers.
- True facts come only from the curated pool; generators never invent health facts.
- Every entry has `review_status: "unreviewed" | "in_review" | "reviewed"` (all `unreviewed` now). Fact items and myths also carry a `basis` (source or rationale), checked by `tests/unit/content-safety.test.ts`.
- `docs/CONTENT_REVIEW.md` is generated from the real banks by `tests/unit/contentReview.test.ts` (`REVIEW_WRITE=1` regenerates it; a plain run fails when the file is stale).

6.2 Types and validation

Content	File	Shape (main fields)	Validation
Fact or Fake item	`content/factOrFake/schema.ts`	`id, format (text, gc_screenshot, image, chatbot_answer), domain, truth, trick, difficulty 1..3, module m01..m10, set (practice, form_A, form_B), pair_id?, author, meta, text, image?, explanation, how_to_check, basis, myth_id?, review_status`	`validateItem`, `validateBank`: id pattern per set, facts use trick `none`, fakes need one of the five tricks, forms need a `pair_id`, image items need a descriptor with alt text, A/B pairs must match in truth, domain, format, trick and difficulty, forms balanced 10 fact and 10 fake
Myth	`content/library/myths.ts`	`id, topic, myth (< 70 chars), fact (< 120), why (<= 40 words), check (<= 25 words), villain, module, basis, program, review_status`	unit tests (`library-content.test.ts`), including stigma-word checks on HIV cards
Glossary term	`content/library/glossary.ts`	`id, term, meaning (<= 15 words), example ("Example: ..."), module, review_status`	unit tests
Comics episode	`content/library/komiks.ts`	episode with 4 to 6 panels, each with scene, characters, up to 2 bubbles	unit tests
Module info	`content/modules.ts`	`id, n, title, short, group (core or more), topic, game, objectives, cards, review_status`	typed
Module content	`modules/types.ts`	`hook, cards[], game, takeaway` (card types in section 7.2)	`modules-engine.test.ts` and per-module tests
Villains and tactic posts	`content/tactics.ts`	five villains, 26 tactic posts, 6 legit posts	unit tests
Badges	`content/badges.ts`	20 badges	typed
Arcade games	`content/games.ts`	slug, name, unlocking module, topic, how, icon	typed

6.3 Item banks and counts

Measured on the repository snapshot:

Bank	File	Count
Fact or Fake practice items	`factOrFake/practice.ts`	64 (32 fact, 32 fake; formats: 21 text, 19 GC screenshot, 13 chatbot answer, 11 image)
Fact or Fake test Form A	`factOrFake/assessment.ts`	20 (10 fact, 10 fake)
Fact or Fake test Form B	`factOrFake/assessment.ts`	20, matched pair by pair to Form A
Library myths	`library/myths.ts`	88 (dengue 13, everyday 13, online 12, medicines 10, chronic 9, vaccines 8, HIV 8, health AI 8, outbreaks 7)
Myths by DOST-NICER program tag (internal)	same	HealthPH+ 17, AIKasalig 16, Mosquito 14, AdvocAid 8, none 33
Glossary terms	`library/glossary.ts`	35
Comics episodes	`library/komiks.ts`	10 episodes, 50 panels ("Season 1: The Miracle Tea")
AI or Not? pool (Module 1 game)	`aiOrNot.ts`	28
Turuan si AI posts (Module 2 game)	`turuan.ts`	10 training, 8 test, 6 local-language
Chatbot rounds (Module 3 game)	`chatbot.ts`	19 rounds, 3 rulings, 1 privacy scenario
Spot the Tactic (Module 4 game)	`tactics.ts`	26 tactic posts, 6 legit posts, 5 villains
Tita's GC scenarios (Module 5 game)	`gc.ts`	5 scenarios, 5 members
Badges	`badges.ts`	20
Arcade games	`games.ts`	11
Lesson cards	`modules/m01..m10`	81 cards including hooks (9 in M1, 8 in each other module), plus 10 takeaway cards

The five villains (tricks) are Dr. Fake (fake expert, deepfaked doctor), Miracle Mike (miracle cure), Fear Monger (fear and urgency), Conspiracy Kuya ("they are hiding the truth") and Kwento Kim (one anecdote as proof).

6.4 Fact or Fake: practice versus test forms (isolation)

The test forms A and B are a matched pre-test and post-test used by facilitators in community workshops to measure growth. They must never be seen in play, so the code isolates them in several layers:

1. **Separate files and chunks.** Practice rounds import `practice.ts` only; the `#/assess/` route imports `assessment.ts` through `assessRound.ts`. `items.ts` (both banks together) exists only for tests and content checks. The release checklist verifies that the test forms (the `assessRound` chunk) are in no practice chunk.
2. **Round builders.** `practicePool()` filters to `set === 'practice'`; mini rounds (5 items, 10 in Module 4), free play (10 items) and the Daily Challenge (3 date-seeded items, at least one fact and one fake) draw from it only.
3. **No UI path.** `#/assess/form_A` and `#/assess/form_B` exist as hidden routes and are never linked. Assessment mode gives binary answers, no per-item feedback until the end, no XP and no sound.

4. **Paper forms** are generated only for facilitators into the git-ignored `print-out/facilitator/<lang>/` and never copied to `public/`, the manifest or the service worker (section 13.4).
5. **Tests:** `fof-bank.test.ts`, `content-safety.test.ts` (keeps the forms out of every practice import path), `printables.test.ts`.

6.5 Fictional-content and FAKE-stamp rules

- Before a verdict, every Fact or Fake card carries a small persistent **"ARAW GAME"** watermark so screenshots cannot circulate as real posts without giving away the answer.
- Every fake in a **revealed** state (feedback card, generated fake post, scam script, fake poster, voice-clone transcript, deepfake example in a lesson) shows the stamp **"ARAW GAME · FAKE"** (red 4 px pixel border, Press Start 2P, rotated -8 degrees). Strings such as `tt.stamp`, `gc.stamp` and `g.sino.stampFake` in `src/app/i18n.ts` carry it; chatbot unsafe answers use `ARAW GAME · UNSAFE ANSWER`.
- "ARAW GAME" is a proper name and is not translated.
- True facts never get the FAKE stamp.

6.6 Health-safety rules in code

- A content-wide unit test rejects any string giving dosing.
- The Fact or Fake "Based on" line shows only public sources (WHO, DOH, FDA, laws); internal references stay in the data for reviewers.
- HIV content follows person-first, stigma-free language rules (`docs/CONTENT_GUIDE.md` section 10), enforced by `library-content.test.ts`.

SECTION 7

Learning modules and the lesson/card engine

7.1 Module flow

Each module (12 to 15 minutes) is plain data plus a lazy-loaded module game. `buildSteps()` in `src/modules/engine.ts` produces the steps:

`hook (story)` → `lesson cards` → `module game` → `"When should I trust it?" takeaway` → `mini Fact or Fake` → `reward`

Rules (all in `engine.ts`, unit tested):

- Modules unlock **sequentially**; M1 is open. Finished modules can be replayed with no repeated card XP.
- Progress is saved after every card; leaving mid-module resumes at the same card.
- **Module games never block progress.** Finishing always counts; stars (1 to 3) show mastery and "Try again" is offered. A failed game load offers Retry; a failed practice round offers Skip. Seniors must never get stuck.
- XP: lesson card 5 (first time only), module game 50, each correct practice item 10 (once per item per module), module complete 100. Each reward is keyed in `xpClaims`, so replays pay nothing twice.
- Finishing a module awards its badge, unlocks the next module and the matching Comics episode, and (M6 to M10) the module's game in the Arcade; finishing all ten awards the "Full Day" badge and the certificate.
- Every lesson card and game screen has a Report flag carrying the card or item id.

7.2 Card types

Type	Purpose	Interaction
`story-hook`	cast scene plus one line (speaker, line, scene)	tap to continue
`info`	one idea, at most about 60 words	tap Next
`reveal`	2 to 6 tiles with hidden detail	tap tiles to reveal
`choice`	one question, 2 to 4 options, one correct; feedback ("why") for every option	try again until correct
`swipe-sort`	4 to 8 mini-cards into two bins	swipe on touch; buttons and arrow keys everywhere
`poll`	opinion question without a right answer	friendly reflection after answering
`takeaway`	"When should I trust it?" in three bullets	read

Each card may carry Ara lines (`intro` , `done`). Ara's sprite mood follows the outcome (correct: happy; wrong: think; badge: wow then happy).

7.3 The ten modules

M1 to M5 are the Core row; M6 to M10 are the More row on the same Learn path.

Id	Title	Topic	Cards	Module game (slug)	Arcade game that unlocks
m01	What Is AI, Really?	what AI is, AI around us, real AI versus hype	9	AI or Not? (`ai-or-not`)	

Id	Title	Topic	Cards	Module game (slug)	Arcade game that unlocks
m02	How AI Learns and Makes Mistakes	training data, bias, confident errors	8	Turuan si AI (`turuan-si-ai`)	
m03	AI and Your Health	chatbot answers, red flags, health data privacy	8	Tanong kay Chatbot (`tanong-kay-chatbot`)	
m04	Tricks of Health Misinformation	five tricks, deepfaked doctors, trusting true posts	8	Spot the Tactic (`spot-the-tactic`); 10-item Fact or Fake with Truth Radar	
m05	Check Before You Share	three-step checking habit, replying kindly, reporting	8	Tita's GC (`titas-gc`)	
m06	AI as Your Partner	AI at work, human in the loop, responsibility	8	Hati Tayo! (page version)	Hati Tayo!
m07	Made by AI?	generative AI, labels, consent, credit	8	Gawa Ko 'To! (3 briefs)	Gawa Ko 'To!
m08	Take a Break	attention design, sleep, AI companions	8	Tulog na, Bea! (3 days)	Tulog na, Bea!
m09	For Everyone	who is left out, bias from data, inclusive design	8	Sa Sapatos Nila (5 visitors)	Sa Sapatos Nila
m10	Your Rights	data rights, complaint path, accountability	8	OBJECTION! (3 cases)	OBJECTION!

Module games for M1 to M5 stay available as Practice replays once their module is complete. Each `src/modules/mNN/index.ts` exports a `ModuleContent` ; `src/modules/registry.ts` lazy-loads it (and the mini Fact or Fake slot).

SECTION 8

Game engine and arcade games

8.1 Shared engine (src/games/engine/)

File	Role
types.ts	GameDef contract: slug, name, logical width and height, goodRun threshold, howTo, five-page briefing, takeaway, create({seed, rng, cue}) returning a GameInstance with step(), draw(), stats()
loop.ts	fixed 60 Hz time step (STEP_MS); frames longer than 250 ms are clamped so the game never "catches up" in a burst
scale.ts	letterbox fit of the logical size into the available box; whole-number device-pixel scaling for crisp pixel art; handles devicePixelRatio, resize and orientation change mid-game
input.ts	one model for keyboard, mouse and touch: actions (left right up down act), taps, pointer position, swipes
rng.ts	seeded randomness (section 8.3)
GameShell.tsx	HUD, pause (also when the tab is hidden), How to play, first-run briefing, game over with recap and XP, text description of game state for screen readers
Briefing.tsx	Ara's paged, skippable first-run briefing (5 pages)
session.ts	PlayClock : after 10 minutes of cumulative play in a session, Ara pauses for a "Pahinga muna!" break suggestion
cues.ts , draw.ts , useRunEnd.ts	sound cues (never per frame), canvas text and sprite helpers, end-of-run sound for DOM games

Game logic is pure and silent in unit tests (sound is passed as a cue callback the shell supplies). Games are pixel art with image-rendering: pixelated .

8.2 The eleven arcade games

Canvas games are registered in GAME_LOADERS and DOM games (real page elements) in DOM_GAME_LOADERS (src/games/registry.ts); each is its own lazy chunk. Several games have **two faces** sharing one rules file: the arcade version (endless, seeded) and the module version (short, always finishable).

Slug	Game	Topic	Type	Unlock	Mechanic	Input
takbo-data	Takbo, Data!	data privacy	canvas	open	endless runner across the barangay; grab safe freebies, avoid data traps; Privacy Shield meter	Space, tap or click to jump; Up/Down or W/S or swipe to change lane
feed-mo-to	Feed Mo 'To	algorithms, filter bubbles	canvas	open	you are the feed algorithm: pick 1 of 3 posts before a timer; scary posts raise Engagement but shrink the bubble	arrows or A/D then Space or Enter; tap or click a post
luto-ni-ara	Luto ni Ara	prompting AI well	canvas	open	cook an answer: add the prompt ingredients the order names, then COOK; 3 hearts	tap, click or drag ingredients; arrows then Space or Enter

Slug	Game	Topic	Type	Unlock	Mechanic	Input
`sino-to`	Sino 'To?	voice-clone scams	canvas	open	read call transcripts, run verification checks, then HELP or HANG UP + REPORT; fictional "PisoPay" and "Banko Liwanag"	tap or click; arrows then Space or Enter
`lamok-lamay`	Lamok Lamay	how AI sees and misses	canvas	open	whack real larvae the AI boxed, add unboxed ones, reject wrong boxes; Precision and Catch-rate meters	tap or click; arrows to pick, Up/Down to switch tool
`tawid-lola`	Tawid, Lola!	when to trust the AI guide	canvas	open	help Lola cross lanes; an AI guide says GO or WAIT and is sometimes confidently wrong; Trust Meter	GO/WAIT tiles, arrows or WASD, Space or Enter; tap
`hati-tayo`	Hati Tayo!	AI and jobs	canvas	M6	split a day's task queue between the AI lane and the person; Service meter	tap task then lane; drag; Up/Down then Left/Right
`gawa-ko-to`	Gawa Ko 'To!	generative AI and consent	DOM	M7	make health posters with AI tools the honest way: label, ask consent, credit	buttons and form controls
`tulog-na-bea`	Tulog na, Bea!	digital wellbeing	DOM	M8	life sim balancing Bea's sleep, mood, study and friends	buttons
`sa-sapatos-nila`	Sa Sapatos Nila	fairness, accessibility	DOM	M9	use a health kiosk as different people, then fix it for everyone	buttons and keys
`objection`	OBJECTION!	your rights with AI	DOM	M10	spot the false statement at the barangay hearing and present evidence	buttons

The canvas logical sizes and "good run" thresholds are in each game file (for example `goodRun` is 400 for Takbo, Data!, 1200 for Feed Mo 'To, Luto ni Ara and Sino 'To?, 2000 for Lamok Lamay, 1000 for Tawid, Lola! and 500 for Hati Tayo!). A test-only `demo` game exists for engine tests and is never listed.

8.3 Randomness and seeding

`src/games/engine/rng.ts` provides a deterministic generator: FNV-1a hashes a string seed to a 32-bit integer and **mulberry32** produces floats in [0, 1). The same seed gives the same sequence, so unit tests are reproducible and the Daily Challenge is identical for everyone. `randomSeed()` gives a fresh seed for normal runs; `dailySeed(date)` returns `ARAW-YYYY-MM-DD`. Helpers: `rngInt`, `rngPick`, Fisher-Yates `shuffle`.

8.4 Scoring, XP, badges and streaks

Topic	Rule	Code
Arcade run XP	10 base, +20 for a good run (score reaches the game's `goodRun`), +20 for a new personal best, maximum 50	`app/scores.ts`
Personal best	stored per game slug in `bestScores`	`scores.ts`
Fact or Fake XP	10 per correct item (none in assessment mode)	`dailyChallenge.ts`, `modules/engine.ts`
Fact or Fake scoring	signal detection: hit rate H and false-alarm rate F with the log-linear correction `(x + 0.5) / (n + 1)`, `d' = z(H) − z(F)`, bias `c = −(z(H) + z(F)) / 2`, with "fake" as the signal; `z` is the inverse normal CDF (Acklam approximation)	`fact-or-fake/scoring.ts`

Topic	Rule	Code
Truth Radar	two plain meters: discernment (`d'` clamped 0..3) and lean (`c` clamped -1..1; within 0.25 is "Balanced"); rounds under 10 items show a "rough read"	`TruthRadar.tsx` , `results.ts`
Daily Challenge	seed `ARAW-YYYY-MM-DD` on the Asia/Manila date; featured game = (days since 2026-01-01) mod 11 over the 11 arcade games; plus 3 Fact or Fake practice items; one scored attempt per day; +30 XP bonus when both parts are done; Wordle-style share text (Copy or Web Share API, nothing sent by the app)	`daily.ts` , `dailyChallenge.ts`
Streak	a day counts on any real activity (card, game run, Fact or Fake round, daily); one automatic **freeze** per Monday-to-Sunday week bridges exactly one missed day; otherwise restart at 1 with a kind message; a clock moving backwards changes nothing	`streak.ts` , `progress.ts` (`recordActivity`)
Badges (20)	10 module badges, 5 villain badges (Spot the Tactic), Balanced Radar, streak badges at 3, 7 and 30 days, Full Day (all 10 modules)	`content/badges.ts`
Certificate	after all 10 modules; the player types a name (not stored), previews, then prints (A4 landscape) or downloads a PNG drawn on canvas	`screens/certificate/`

Dates always use the **Asia/Manila** time zone (`Intl.DateTimeFormat` with calendar arithmetic in UTC, so no daylight-saving effects).

SECTION 9

Internationalisation

9.1 Languages and design

ARAW ships in **five languages**: English (source), Filipino (`fil`), Cebuano (`ceb`), Ilokano (`ilo`) and Hiligaynon (`hil`), the five languages of the AIKasalig project. English is the source language in the code; the other four are **runtime packs**: `src/i18n/packs/<code>.json` , a flat map from the exact English sentence to its translation. A language appears on the pickers as soon as its pack file exists (`hasPack()` uses `import.meta.glob`).

Why this design: no screen, content file or game needed to be rewritten for translation; strings missing from a pack simply show in English (a partly translated language never breaks the app); and only the chosen language's pack is downloaded (packs are lazy chunks, not precached).

9.2 Runtime

Piece	File	Behaviour
`t(key, vars)`	`src/app/i18n.ts`	the UI catalog (1,996 keys in the English source) with `{placeholder}` substitution; the English text is then passed through the active pack
`tx(text)`	`src/i18n/runtime.ts`	exact-match lookup in the active pack; a quoted line is matched by its inner sentence; unknown text passes through unchanged
`loadPack(code, remount)`	`runtime.ts`	loads the pack, never throws (falls back to English); bumps `packVersion` so the app remounts in the new language
Render hook	`src/i18n/dom.ts`	a Preact `options.vnode` hook translates string children and text attributes (`alt` , `title` , `placeholder` , `aria-label` ...) as elements are created, so content arrays (myths, items, lessons) render translated from their English source; elements with `data-no-tx` (player text, AI answers) are left untouched
Canvas text	`dom.ts`	the canvas text calls are wrapped so games drawing words on a canvas are translated too
Language state	`src/app/language.ts`	saved choice (`araw:v1:language`); `<html lang>` kept in sync for screen readers
QA aid	`runtime.ts`	`?i18n-missing` in the URL collects English-looking strings with no translation in `window.__arawMissing`

Proper names stay as they are in every language: ARAW, Ara, game titles, characters, fictional places, pages and brands, the "ARAW GAME" part of stamps, hotline numbers and URLs. Module 2's detector game deliberately keeps its Taglish and Cebuano example posts, because the lesson is that an English-only AI misreads local languages.

9.3 Extraction and translation pipeline (`scripts/i18n/`)

The pipeline is resumable at every step and is documented in `scripts/i18n/README.md` ; translators follow `TRANSLATOR.md` and a per-language glossary in `glossary/<code>.md` .

Step	Command	Result
1. Extract	`node scripts/i18n/extract.mjs`	refreshes `source.json` : every player-facing English string from the UI catalog and every exported value in `src/content` , `src/modules` , `src/games` , `src/screens` , `src/ui` (loaded through Vite and walked deeply; ids, file names and code-like strings are skipped). It also reads string literals in `scripts/print/page/**` , so print framing text and lecture captions use the same pipeline

Step	Command	Result
2. Batch	`node scripts/i18n/batches.mjs <code>`	splits what is still untranslated into `work/<code>/in/bNNN.json` (150 lines each)
3. Translate	write `work/<code>/out/bNNN.json`	done batches are never redone; an interrupted run just continues
4. Pack	`node scripts/i18n/pack.mjs <code>`	checks that `{placeholders}` match, merges hand-made `extra.json`, and writes `src/i18n/packs/<code>.json`. Bad entries are left out (the string stays English) and listed in `work/<code>/report.txt`
5. Health review (optional)	`node scripts/i18n/review.mjs <code> [size]`	queues health-critical lines (myths, Fact or Fake items and forms, lessons, chatbot and group-chat rounds, tactics, glossary) with the English and the current translation for a reviewer; `pack.mjs` applies corrections
6. Verify	`npm run build && npx playwright test tests/e2e/i18n.spec.ts`	every pack's main screens must fit at phone, tablet and desktop; untranslated English seen on screen is listed in `work/<code>/missing.txt`

After content changes, repeat steps 1 to 4: only new or changed strings become new batches. A unit test also fails if a how-to video storyboard caption is missing from a pack (decision log, TUTORIAL-PIPELINE).

9.4 Coverage per language (committed snapshot, commit `234b875`)

Code	Language	Pack entries	Coverage of the committed source list (6,509 strings)	Translated by	Native review
`en`	English	source	source	authors	n/a
`fil`	Filipino	6,572	100%	Claude Sonnet; Opus review of health lines pending	needed
`ceb`	Cebuano	6,572	100%	Claude Sonnet; Opus review of health lines pending	needed
`ilo`	Ilokano	6,572	100%	Claude Opus	needed
`hil`	Hiligaynon	6,572	100%	Claude Opus	needed

Each pack is about 0.84 to 0.86 MB of JSON before compression. The translations were machine-produced (batch status in `scripts/i18n/README.md`: 2026-10-16, with later batches up to b051 for project identity, videos, sandbox and print) and **must be checked by a native speaker and the health reviewer before a public launch**, especially numbers, ages, time spans, warnings, hotline and dose-related lines. The packs grow when new text is added: a user-guide update that was in progress while this document was written adds further batches (b052 and b053) and regenerates the packs, so entry counts after that commit are higher than the table shows.

SECTION 10

Accessibility implementation

The target is **WCAG 2.2 AA**, always on. Measures found in the code and tests:

Area	Implementation
Semantics	real `button`, `a`, `label` and `input` elements; landmark and heading structure; a skip link that moves focus to `#main`; focus moves to the content when the splash closes and to the splash when the language landing closes
Focus	a visible 3 px ink focus ring on all interactive elements (`:focus-visible` in `base.css`), driven by the `--focus` and `--focus-w` tokens
Target size	tap targets of at least 44 px; the "Extra-large buttons" setting enlarges further
Contrast	text at 4.5:1 or better; coral is used for fills only, never text; a dedicated `--fact-ink` token exists because `--fact` alone is 4.48:1 on its tint; "High contrast" setting
Text	body 18 px Nunito, minimum 12 px pixel text; "Larger text" scales through `--text-scale`
Motion	step-based animations; `prefers-reduced-motion` and the in-app "Reduce motion" setting both disable motion (confetti becomes a static sparkle, the splash completes at once)
Text alternatives	alt text or ARIA on every image, icon button and game state; pixel scenes carry descriptors with alt text; videos have WebVTT captions
Games	the canvas has `role="img"` with an `aria-label` replaced by a plain-words description of the current game state; `GameShell` has a polite live region, a text HUD (`aria-label`ed list), a recap read at game over, and keyboard, mouse and touch input for every game
Screen readers	the "Screen-reader hints" setting adds extra spoken guidance; `<html lang>` follows the chosen language
Sound	every sound has something on screen to match; Music and Sound effects switches and volumes; autoplay is attempted only after the language tap, with a first-interaction fallback
Reading level	content aims at Grade 6 to 8, sentences under about 20 words, feedback at most 40 words

Tests: `@axe-core/playwright` scans run in 31 of the 37 e2e spec files, with the tags `wcag2a`, `wcag2aa`, `wcag21a`, `wcag21aa` and `wcag22aa`; 34 spec files assert no horizontal overflow. `scripts/audit/audit.spec.ts` runs a heuristic layout audit (overflow, clipped text, small targets) across the 5 languages and 5 viewports (phone, phone landscape, tablet, 1366 and 1920 desktop). `tests/e2e/game-text-fit*.spec.ts` tours the games and measures canvas and DOM text against its boxes in every language (not part of verify). Automated checks do not replace a manual screen-reader review, which is still advisable before a public launch.

SECTION 11

Offline and PWA

11.1 Service worker generation

There is no PWA plugin dependency. `scripts/pwa-plugin.ts` (a Vite plugin, build only) emits `sw.js` from `scripts/sw-template.js` in `generateBundle`, filling two placeholders:

- the **precache list**: every emitted file in the Vite bundle **except** old `.woff` fallbacks, **language packs** (`src/i18n/packs`) and the **Chatbot Sandbox engine** (any chunk that includes `@wllama` or `src/sandbox/wllama*` or `fake*`, and every `.wasm` asset); plus `./`, `index.html`, `favicon.svg`, `manifest.webmanifest` and the five icon files;
- the **cache name** `araw-<version>-<8 hex characters of the SHA-1 of the list>`, so any change to a hashed file produces a new cache and `activate` deletes older `araw-*` caches.

`src/app/pwa.ts` registers `./sw.js` with scope `./` in production builds only, after `load`; failures (private mode, plain HTTP) leave the app online-only.

11.2 Fetch strategy

Request	Strategy
Non-GET or cross-origin (for example Hugging Face)	not handled; goes to the network
Paths containing `/videos/` or `/print/`	not handled; always network (range requests for video, on-demand PDFs)
Navigation	network first, so a new version appears; the cached `index.html` answers when offline
Everything else (same origin)	cache first (`ignoreVary: true`, because static hosts may send `Vary: Origin`); a miss is fetched and stored in the current cache, so the chosen language pack, the sandbox engine and the guide pictures are kept after first use

`install` precaches and calls `skipWaiting`; `activate` calls `clients.claim()`. The manifest (`public/manifest.webmanifest`) sets `display: standalone`, `start_url ./#/`, scope `./`, theme colour `#F5D89E`, background `#FFF8EC`, categories education and health, with 192, 512, maskable-512 and SVG icons generated by `scripts/make-icons.mjs` from a 16 by 16 pixel sun mark.

11.3 What is downloaded on demand (never precached)

Item	Where it lives	When it downloads
Language pack (about 0.85 MB each)	lazy JS chunk	when that language is chosen; then runtime-cached
How-to videos (35 MB in total on disk; 13 clips x 5 languages, about 0.4 to 1.3 MB each)	`public/videos/<id>/<lang>.webm` and `.vtt`	streamed when played
Print PDFs (about 11 MB in total; 50 files)	`public/print/<lang>/<id>.pdf`	when the player taps Download
User-guide pictures	`public/guide/*.webp`	when the Guide screen shows a section
Chatbot engine (wllama scripts and two `.wasm` files, 16.0 MB and 8.8 MB in the current build)	lazy chunks	first use of the sandbox (or the background preload), then runtime-cached

Item	Where it lives	When it downloads
Chatbot model (378 MB for the default)	Hugging Face, stored in OPFS by wllama	background preload or first sandbox visit (section 12.4)

The sandbox *screen* chunk itself is precached.

SECTION 12

Chatbot Sandbox

12.1 Purpose and architecture

The sandbox (`#/sandbox`) lets a learner chat with a **real, small AI that runs on their own device**, see where it goes wrong, and practise calibrated trust on fresh answers. It is the live counterpart of the scripted Module 3 game. There is no server and no API key, so any number of people can use it at once at no cost, and conversations never leave the device (decision log, SANDBOX).

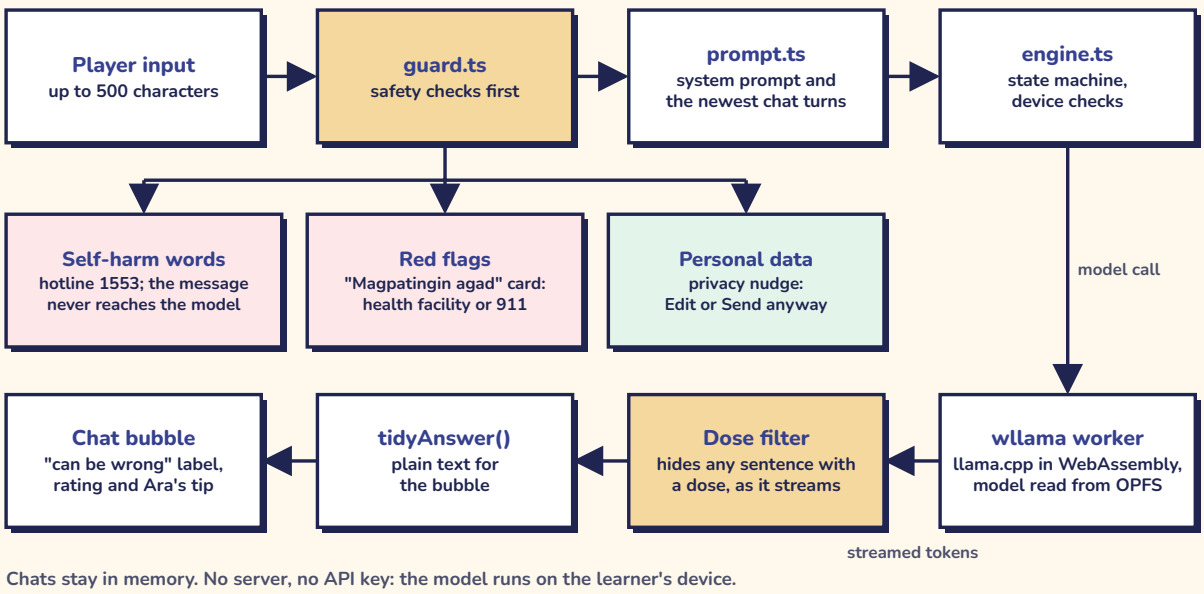


Figure 2. Chatbot Sandbox: the path of one message. Safety checks run before the model; the dose filter runs on the streamed answer.

Module	Role
`engine.ts`	state machine (`idle, checking, unsupported, downloading, loading, ready, error`) over a `Backend` ; device checks; `init` , `load` , `prefetch` , `chat` , `stop` ; one engine per model, kept while the app is open
`wllamaBackend.ts`	the real backend: wllama 3.8.1 (MIT) in its own worker; GPU through WebGPU or several CPU threads when the browser allows; model files in the browser's private file storage (OPFS); all engine files are served by ARAW itself
`wllamaCompat.ts`	wllama's compatibility build for browsers without JSPI or 64-bit WebAssembly memory (mostly Safari); its own lazy chunk; used when the browser lacks those features and is not Firefox (Firefox runs the normal build without the GPU)
`fakeBackend.ts`	scripted backend for tests, selected by `window.__ARAW_FAKE_LLM__` ; no test downloads a model (the e2e fixtures also abort `huggingface.co`)
`model.ts`	pinned model table, `DEFAULT_MODEL_ID` , `N_CTX = 2048` , `MAX_TOKENS = 320` (small enough for phones with 3 to 4 GB of memory), model URL builder
`prompt.ts`	system prompt and history budget
`guard.ts`	safety checks (section 12.6)
`missions.ts`	"Catch the AI" missions

Module	Role
`preload.ts`	quiet background download (section 12.4); tiny, part of the first load
`review.ts`	list of sandbox lines for the reviewers

Generation settings: temperature 0.6, top-p 0.9, top-k 40, repeat penalty 1.1, streamed, abortable (Stop button).

12.2 Model hosting and pinning

The model is the one approved exception to "no CDNs at runtime". It is fetched from a **pinned Hugging Face commit**, so the file can never change under the project:

`https://huggingface.co/<repo>/resolve/<revision>/<file>`

For the default: repo `unsloth/Qwen3-0.6B-GGUF`, revision `50968a4468ef4233ed78cd7c3de230dd1d61a56b`, file `Qwen3-0.6B-Q4_K_M.gguf`. Why this hosting: Cloudflare Pages caps files at 25 MiB and model weights do not belong in Git. wllama's own `.wasm` and worker files are bundled and served by ARAW (the largest, wllama's 16 MB compat `.wasm`, stays under the limit).

Switchable source. `VITE_MODEL_SOURCE=local` at build time makes the app load `./models/<file>` (next to `index.html`) instead of Hugging Face. `node scripts/sandbox/fetch-model.mjs [modelId]` downloads the pinned file into git-ignored `public/models/` and checks its SHA-256 against Hugging Face's LFS etag. This is the hook for the planned offline desktop and Android apps (section 18.2).

12.3 Device checks

`supportProblem()` blocks a device only for a known reason; fields a browser does not report never block:

Check	Fails when	Reason shown
WebAssembly with SIMD and exception handling (probed with small test modules) and Web Workers	missing	`browser`
`navigator.deviceMemory`	below 2 GB	`memory`
`navigator.storage.estimate()` free space	below 1.2 x the model size	`storage`

A blocked device gets a kind explanation and a link to Module 3 (the scripted version); there is no scripted fallback inside the sandbox (user decision, SANDBOX-LOAD).

12.4 OPFS cache and background preload

- The model file is cached by wllama's model manager in OPFS. Later visits load it from the device, **even offline**. `isCached()` compares the stored model URL with the pinned URL.
- **Preload rules** (`preload.ts`, `okToPrefetch()`): about **8 seconds** after the app loads, when the browser is idle (`requestIdleCallback`, 10 s timeout), the engine downloads the file **only to storage, never into memory**, so lessons and games stay smooth. It runs only when the connection is unmetered or unknown: it is skipped on `saveData`, on connection types `cellular`, `bluetooth`, `wimax` and `none`, and on 2G effective types; iPhones and desktops, which do not report a type, do download. **Automated browsers** (`navigator.webdriver`) and the test fake never download.
- It also does nothing when the device cannot run the model, when the file is already stored, or once the sandbox has started; failures stay silent. Opening the sandbox continues a running download and shows its progress ("This happens only once"); on mobile data the download starts when the sandbox opens.

- Loading into memory happens only when the sandbox opens.

12.5 Cross-origin isolation headers

`public/_headers` (Cloudflare), the Hugging Face Space `custom_headers` (README front matter) and the Vite dev and preview servers send:

```
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: credentialless
```

This makes the page cross-origin isolated, so wllama can use `SharedArrayBuffer` and several CPU threads. `credentialless` (rather than `require-corp`) still allows the model download from Hugging Face. GitHub Pages cannot send custom headers, so there the AI runs on one thread (slower, still works).

12.6 Safety layer (`guard.ts`, five languages)

All checks are pure functions with unit tests (`sandboxGuard.test.ts`). Word lists are deliberately broad, because a false alarm costs one tap and a missed red flag could cost much more. Input is lower-cased, accent-stripped and normalised first; terms of four letters or fewer must match a whole word (so "aso" does not match inside "kaso").

Safeguard	Behaviour
Red flags	ten rule groups (chest pain or heart attack, breathing trouble, stroke signs, seizure or fainting, heavy bleeding, dengue warning signs, dehydration in a child, fever in a baby, animal bite, self-harm) matched in English, Filipino, Cebuano, Ilokano and Hiligaynon. A hit shows the "Magpatingin agad" card (go to the nearest health facility or call 911) **before any answer**; the player may still ask "in general" with one more tap
Self-harm	shows the crisis hotline (DOH-NCMH **1553**) and the message **never reaches the model**
Privacy nudge	detects names, Philippine mobile numbers, long ID-like numbers, emails, addresses and diagnoses; a nudge explains that real chatbots may store this and offers Edit or Send anyway (it stays on the device)
Dose filter	`createDoseFilter()` checks the **streamed answer sentence by sentence**: any sentence with a number plus a medicine unit, a number word plus a unit, "x times a day", "every N hours" or their translations, or "dosage" is replaced by "ARAW hid a medicine dose here"; the unfinished last sentence is shown only up to its first digit, so "500 mg" never flashes on screen
System prompt	English instructions (small models follow them best) naming the answer language: at most 5 sentences, admits it can be wrong and does not know the date or news, never gives doses, diagnoses or treatment changes, points to a health worker, never asks for personal details
Framing	every answer is labelled "ARAW practice AI · can be wrong" and gets a Usable / Wrong / Get checked rating with a calibrated-trust tip from Ara; generated text is never presented as a fact source; chats are kept in memory only; input is capped at 500 characters

`src/sandbox/review.ts` lists nine sandbox elements (system prompt, starters, red flags, self-harm, red-flag card, personal data, dose filter, trust tips, missions) for the medical and cultural reviewers. The model's own answers are generated and cannot be reviewed.

12.7 Missions

Five "Catch the AI" missions, **+10 XP each, paid once** (`xpClaims` key `sandbox:<id>`); no new badge, because the badge set is fixed at 20.

Id	Task	Completion
`ask`	ask a first question	automatic

Id	Task	Completion
`date`	ask the date (the model does not know it)	ticked by the player
`source`	ask for a source or link	ticked by the player
`language`	ask in another language	ticked by the player
`rate`	rate 3 answers	automatic (`RATINGS_NEEDED = 3`)

The route is `#/sandbox` (Arcade tab, study music), linked from the Arcade Practice row and from the Tanong kay Chatbot start and end screens. It is **not** in `ARCADE_GAMES`, which drives the Daily Challenge, the Home counts and the tutorial list.

SECTION 13

Media and document pipelines

All pipelines compile from the app's own typed content, so text stays in one place. None needs a server of its own; where a local web server is needed it runs on its own port (4173 for tutorials, 4180 for the guide capture, 4182 for lectures and the text-fit tours) and must be stopped afterwards.

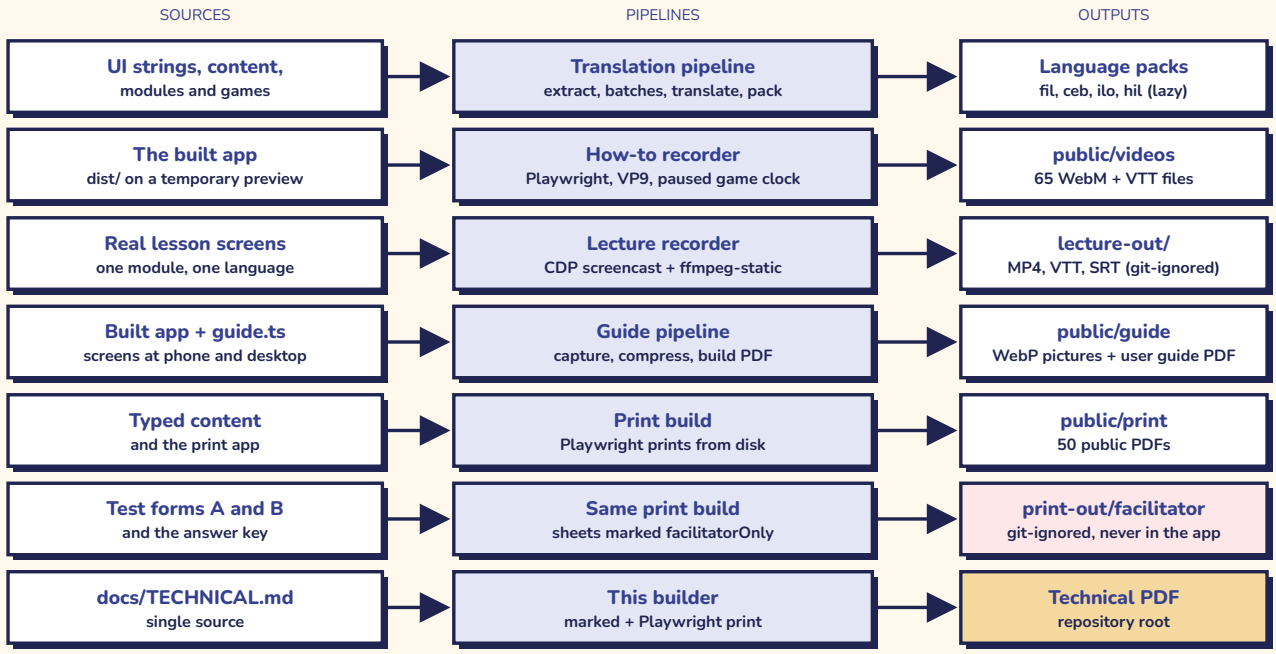


Figure 3. Build and media pipelines. Everything is compiled from the app's own typed content; test forms are printed for facilitators only.

13.1 How-to videos (`scripts/tutorials/`)

13 clips (a tour, Fact or Fake, and 11 arcade games) in 5 languages, short, silent and captioned: **65 WebM files** with matching WebVTT. They are **recorded, not filmed**: `record.mjs` drives the built app with Playwright from one storyboard per clip (`boards/<id>.mjs`), draws a pixel cursor, key caps and title cards into the page, and encodes VP9 at 20 fps with Playwright's bundled ffmpeg (games 1024 x 576, the tour 1280 x 720). Canvas games run on a paused fake clock advanced in 40 ms ticks, so every recording repeats exactly.

```
npm run build                                        # the recorder serves dist/ on port 4173
node scripts/tutorials/record.mjs                    # every clip, every language (about 30 minutes)
node scripts/tutorials/record.mjs takbo-data --langs=en # one clip, one language
node scripts/tutorials/probe.mjs takbo-data          # plan a canvas game's key timeline
# --shots saves a PNG per caption; --dry writes to shots-out/tutorials/dry/
```

Output: `public/videos/<id>/<lang>.webm|vtt` and `src/content/tutorialManifest.json` (length and size per clip; it is what turns on the Watch buttons, so rebuild after recording). Re-record a clip when its screen or translation changes. A step that cannot find its element fails the clip loudly and saves a screenshot.

13.2 Module lecture videos (`scripts/lectures/`)

Silent, slow **H.264 MP4s** of one module, recorded from the real lesson screen at **1920 x 1080** (the 1280 x 720 desktop layout enlarged 1.5 x with CSS zoom), in any of the five languages. A generic driver (`driver.mjs`) plays the title card, objectives, hook, every card (questions get a 5-second "think about it" pause, then the best answer and its feedback), the module game, the takeaway and a closing card with the app address. Holds are paced from reading time (words at 100 words per minute plus 3 s, at most 8 s). Capture uses the Chrome DevTools Protocol screencast; frames are encoded once with `ffmpeg-static` as H.264 High, yuv420p, 30 fps, faststart, no audio.

```
node scripts/lectures/record.mjs m01 --langs=en,fil          # builds dist-lecture/, serves it on port 4182,
records
node scripts/lectures/record.mjs m01 --langs=en --no-build   # reuse an earlier dist-lecture/ build
```

Output per module and language in `lecture-out/<lang>/` (git-ignored): `mNN.mp4` , `mNN.vtt` , `mNN.srt` (one cue per beat) and `mNN-script.md` (a table of beats with times, what is on screen and a narration to read aloud). The videos are silent **so that voiceovers can be added later**, and they stay out of the app until the Library > Videos decision (decision log, LECTURES). Bridge lines are in `src/content/lectures.ts` and use the normal translation pipeline, falling back to English and being listed until translated. A module whose game needs scripted play adds `scripts/lectures/overrides/<mNN>.mjs` ; only `m01.mjs` exists. In the repository snapshot only `m01` , in English and Filipino, has been rendered.

13.3 User guide PDF (`scripts/guide/`)

The guide has two forms from one source: the in-app `#/guide` screen and a printable A4 PDF. Words live in `src/content/guide.ts` ; callout positions are computed by `src/screens/guide/callouts.ts` .

```
npm run build
npx playwright test -c scripts/guide/capture.config.ts   # serves dist/ on port 4180; phone 360x740 @2x and
desktop 1366x768 @1x
node scripts/guide/compress.mjs                           # PNG -> WebP for the app; PNG originals kept in
scripts/guide/out/png
node scripts/guide/build-pdf.mjs [--pages]                # A4 PDF via Playwright Chromium from file://, no
server
```

Output: `public/guide/*.webp` (6.9 MB with the PDF), `public/guide/ARAW-User-Guide.pdf` , and a root copy at the repository root named `ARAW V<version>-User-Guide.pdf` (the version is part of the file name and changes with each guide edition). The PDF builder keeps the PNG originals, because Chromium re-encodes WebP badly inside a PDF.

13.4 Print kit (`scripts/print/`)

The kit is **compiled, not authored** (`docs/PRINT-KIT-PLAN.md`): every sheet is generated from typed app content and translated through the same packs. `scripts/print/page/` is a small Vite and Preact app (one `.tsx` per sheet in `sheets/` , auto-discovered); `build.mjs` builds it into `scripts/print/out/site/` , opens it in Playwright Chromium **without a server** (files are served through `page.route()` from disk) and prints A4 PDFs, one per sheet and language, warning about any overflow.

Sheet id	Content	Format
`trust-guide`	"When should I trust it?": all ten takeaways, QR to the app	A4, 1 page
`module-posters`	one poster per module, QR to `#/learn/<id>`	A4, 10 pages
`five-tricks`	the five villains: trick, tells, counter-move	A4, 1 page

Sheet id	Content	Format
`check-before-share`	the three-step checking habit and a kind reply	A4, 1 page
`red-flags`	"Magpatingin agad" cut-out cards: health facility or 911, hotline 1553	A4, 4 cards
`myth-cards`	myth front and fact back cut-out cards, 8 a page, duplex	A4
`glossary`	plain-words glossary, sorted by the translated term	A4, 2 columns
`komiks`	Season 1 comics, one episode a page	A4
`session-guides`	facilitator guide plus one page per module	A4
`certificate`	blank completion certificate	A4 landscape

Ten **public** sheets x five languages = **50 PDFs** (about 11 MB) in `public/print/<lang>/<id>.pdf`, listed in `src/content/printManifest.json` and offered in Library > Printables (a language picker defaulting to the app language, English fallback per sheet; downloaded on demand, never precached).

Facilitator-only rule. The three sheets `pretest-form-a`, `posttest-form-b` and `answer-key` are the Fact or Fake test sets. They are marked `facilitatorOnly`, are always written to git-ignored `print-out/facilitator/<lang>/`, and are **never** in `public/`, the manifest, the app or the service worker. `src/app/printables.ts` also filters them out of any manifest (`FACILITATOR_ONLY`), with a unit test.

```
npm run print                                       # node scripts/print/build.mjs: every sheet x language
node scripts/print/build.mjs --pages                # also one PNG per page in scripts/print/out/png/<lang>/
node scripts/print/build.mjs --only=red-flags --langs=fil,ilo
node scripts/print/build.mjs --app-url=https://...  # QR target; or env PRINT_APP_URL
```

QR codes point to the live app (default `https://dost-nicer-araw.mjmagtira.workers.dev/`). Print rules: A4 with 12 mm margins and nothing important within 15 mm of an edge. New framing text goes through `tx()` or `t()` and the i18n pipeline; overflow warnings mean a translation is too long for its box.

13.5 This document (`scripts/techdoc/`)

`docs/TECHNICAL.md` is the single source. `scripts/techdoc/build-pdf.mjs` renders it with `marked` to A4 HTML with the ARAW look (cover with partner logos, table of contents with page numbers, footer "page n / N") and prints it with Playwright Chromium from `file://` (no server, no port) to `ARAW V1.0-Technical-Documentation.pdf` at the repository root. Figures marked `<!-- figure:name -->` followed by a text block are replaced by SVG diagrams in the PDF.

```
npm run techdoc                                 # node scripts/techdoc/build-pdf.mjs
node scripts/techdoc/build-pdf.mjs --pages      # also PNG previews in scripts/techdoc/out/ (git-ignored)
```

SECTION 14

Privacy and security

14.1 Privacy by design

Topic	Fact
Accounts	none; no sign-in
Server-side personal data	none stored: there is no backend. The only server-side receiver is the optional report relay (14.2)
Analytics and trackers	none in the code
Progress	`localStorage` on the device, under `araw:v1:`; "Reset my progress" clears it
Certificate name	typed per use, drawn on a canvas or printed, **not stored**
Chatbot	conversations in memory only; the model and the text it sees run on the device; privacy nudge for personal details
Daily share text	copied or shared by the player through the OS share sheet; nothing is sent by ARAW
Health data	the app never asks for health information; the Report screen reminds people not to include any
Third-party requests	only the pinned model file from Hugging Face (sandbox, after the player's use or the Wi-Fi preload), and the optional relay POST

14.2 Report relay (`tools/report-relay/`)

The Report screen offers four ways to report: **Send report** (in-app, only if `VITE_REPORT_ENDPOINT` was set at build time), Open in Gmail, Use my email app (`mailto:`), and Copy report. With no relay configured the Send button is hidden and the fallbacks still work.

The relay is a **Google Apps Script web app** (`Code.gs` , no keys or passwords, safe in a public repository) deployed once under the AIKasalig Google account (steps in its README). It emails `AIKasalig@gmail.com` and appends a row to a Google Sheet called "ARAW Reports".

What the app sends (JSON from `buildReportPayload` , posted as `text/plain` to avoid a CORS preflight, `credentials: 'omit'` , 10 s timeout):

Field	Content
`category`	`bug` , `content` , `suggestion` or `other`
`screen` , `item`	the screen and item (card or game id) the report came from, each at most 60 characters, control characters removed
`details`	the text the player typed, at most 1,500 characters
`device`	a short "OS, browser" label parsed from the user agent (for example "Android, Chrome")
`version`	the app version string
`website`	a hidden honeypot field; people never fill it

No name, email address or IP address is added by the app. **Relay protections:** body at most 8,000 characters; strict field validation and length limits; honeypot rejection; shared limits of **5 reports a minute and 90 a day** (a free Gmail account can send about 100 emails a day); short error codes (`daily_limit` , `bad_json` , `mail_failed` and others) that the app turns into a friendly "try another way" message. Using the email fallbacks reveals the player's own email address to the team, which the app states on screen.

14.3 Headers and hosting security

`public/_headers` sets, for all paths: `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, and the COOP and COEP pair of section 12.5. Hashed files under `/assets/*` are `immutable` for a year; `index.html`, `/` and `/sw.js` are `no-cache` so new versions are picked up.

14.4 Threat notes

Threat	Mitigation or status
Cross-site scripting	content is static typed data rendered by Preact (text is escaped); the code has no `innerHTML`, `dangerouslySetInnerHTML`, `eval` or `new Function`; chatbot output is shown as plain text (`tidyAnswer`) and marked `data-no-tx`
Tampered or corrupt `localStorage`	every persisted value passes a validator on read and falls back to defaults
Supply chain	few dependencies, pinned by `package-lock.json`; the model is pinned by commit hash and (for local builds) checked by SHA-256; wllama files are served from the same origin
Spam to the relay	honeypot, size and field limits, 5 per minute and 90 per day
Misuse of fake content	every fake carries the "ARAW GAME · FAKE" stamp when revealed and an "ARAW GAME" watermark before; all content is fictional
Harmful chatbot output	red-flag and self-harm routing, dose filter, short answers, "can be wrong" label; the model is small and will make mistakes by design
Content security policy	**not set** in `_headers`. A recommended hardening step: add a policy allowing `huggingface.co` for the model, `script.google.com` for the relay and `'wasm-unsafe-eval'` for WebAssembly, after testing
Dependency licence	`ffmpeg-static` is GPL but used only on a developer machine; it is not shipped

SECTION 15

Build, quality gates and testing

15.1 npm scripts

Script	Command	Purpose
`dev`	`vite --port 5173 --strictPort`	development server
`build`	`tsc --noEmit && vite build`	type-check, then build `dist/` and `sw.js`
`preview`	`vite preview --port 4173 --strictPort`	serve `dist/`
`test`	`vitest run --reporter=dot`	unit and component tests
`e2e`	`playwright test --reporter=line`	end-to-end tests on three projects
`print`	`node scripts/print/build.mjs`	print kit PDFs
`budget`	`node scripts/check-budget.mjs`	performance budget
`icons`	`node scripts/make-icons.mjs`	PWA icons
`techdoc`	`node scripts/techdoc/build-pdf.mjs`	this document as PDF
`verify`	`npm run build && npm run budget && npm run test && npm run e2e`	the full gate

Ports are fixed (dev 5173, preview 4173, both `--strictPort`), and long-running processes (servers, watchers) are never left running; no script uses a watch mode.

15.2 The verify pipeline

1. **Build:** `tsc --noEmit` (strict) then `vite build`.
2. **Budget** (`scripts/check-budget.mjs`, reads `dist/`):
   - initial JS (entry script plus its `modulepreload` links) at most **200 KB gzip**;
   - initial CSS at most **40 KB gzip**;
   - no module, game or other lazy chunk (names such as `m01` to `m10`, `takbo`, `hati`, `SandboxScreen`, `wllamaBackend`) in the initial load; module and game chunks must exist (code splitting intact);
   - each image or SVG at most 24 KB and all images together at most 120 KB (the `guide` folder is excluded because it loads one section at a time).

   Last measured: initial JS **103.7 KB gzip** in 11 files, CSS **15.5 KB gzip**, 12 images 75.2 KB.
3. **Unit tests:** Vitest in jsdom, `tests/unit/**`.
4. **End-to-end tests:** Playwright against `vite preview` on port 4173 (`reuseExistingServer: false`).

`npm run verify` must be green before every commit (CLAUDE.md working rules).

15.3 Unit and component tests

Vitest 5 with jsdom and `@testing-library/preact`: **1,101 tests in 68 files** (measured). Areas: engine (loop, RNG, scale, input), scoring and Truth Radar, rounds and daily seeds, streaks and XP, store and storage-blocked behaviour, router, settings, language and packs, every game's logic and many components, content banks

(validation, counts, safety, review file), sandbox (guard, engine, model, preload, missions, prompt, screen), report, PWA output, printables, guide content and callouts.

15.4 End-to-end tests

Playwright 1.64 with Chromium, three projects defined in `playwright.config.ts`:

Project	Viewport	Notes
`phone`	360 x 740	mobile emulation, touch
`tablet`	820 x 1180	
`desktop`	1440 x 900	

900 tests (300 per project) in 37 spec files (measured with `playwright test --list`). Every spec runs on all three projects, scans for accessibility with axe (WCAG 2.0, 2.1 and 2.2 A and AA tags; 31 files) and asserts "no horizontal overflow" (`noHorizontalOverflow` in `tests/e2e/fixtures.ts`; 34 spec files). Shared fixtures skip the splash through `sessionStorage`, install the scripted test AI and abort Hugging Face requests. Specs cover each screen and each game, the shell and responsiveness, i18n fit for every pack, PWA behaviour, the sandbox, the tutorials (including "videos are never cached by the service worker"), printables, the guide, sound, and the certificate.

15.5 Tests outside `verify` (run by hand)

Purpose	Command
Release screenshot sweep (every route, module hook and game step at three viewports, every accessibility setting)	`npx playwright test -c scripts/shots/shots.config.ts` (output `shots-out/`)
Layout audit (5 languages x 5 viewports)	`npx playwright test -c scripts/audit/audit.config.ts`
Canvas and DOM game text-fit tours	`npx vite build --outDir dist-canvas && npx playwright test -c tests/e2e/game-text-fit.config.ts` (and `game-text-fit-dom.config.ts`)
Real-model sandbox smoke test and model bake-off (downloads models)	`npx playwright test -c scripts/sandbox/smoke.config.ts`
User-guide capture	`npx playwright test -c scripts/guide/capture.config.ts`

15.6 Test counts at a glance

Suite	Files	Tests
Vitest unit and component	68	1,101
Playwright e2e (3 projects)	37	900 (300 x 3)

SECTION 16

Deployment and configuration

16.1 Hosts

The app is plain static files (`dist/`), built with a relative base so it works at any path. The README documents three targets:

Host	How	Headers
Cloudflare (live site)	connect the GitHub repository; build command `npm run build`, output directory `dist`; every push to `main` deploys (the README describes Cloudflare Pages)	`public/_headers` is honoured: caching plus COOP and COEP
Hugging Face static Space	README front matter sets `sdk: static`, `app_build_command: npm run build`, `app_file: dist/index.html` and `custom_headers` (COOP and COEP)	through `custom_headers`
GitHub Pages	a GitHub Actions workflow (given in the README) builds and publishes `dist/`	no custom headers: the sandbox AI runs on one thread

How the live site is deployed: what the repository settles and what it does not. The live address is a `*.workers.dev` host, which is Cloudflare's address style for Workers (static assets), whereas the README and `_headers` describe Cloudflare Pages. The repository contains no `wrangler` configuration and no CI workflow file, so the exact Cloudflare setup (Pages or Workers static assets, build settings, and the value of `VITE_REPORT_ENDPOINT` in that build) **cannot be confirmed from the code**. The maintainer should record the deployed configuration here.

Every file must stay under Cloudflare's 25 MiB per-file limit (the largest is wllama's 16 MB compat `.wasm`).

16.2 Environment variables

Variable	Read by	Meaning	Default
`VITE_REPORT_ENDPOINT`	the app at build time (`src/app/report.ts`)	Google Apps Script `/exec` URL of the report relay; empty hides the Send button. Not a secret (it ships in the built JS); may be committed in `.env.production` (see `.env.example`)	empty; no `.env.production` is committed in the repository
`VITE_MODEL_SOURCE`	the app at build time (`src/sandbox/model.ts`)	`local` loads the model from `./models/` instead of Hugging Face	`hf`
`PRINT_APP_URL`	`scripts/print/build.mjs`	URL encoded in the QR codes	`https://dost-nicer-araw.mjmagtira.workers.dev/`
`REVIEW_WRITE=1`	`tests/unit/contentReview.test.ts`	regenerate `docs/CONTENT_REVIEW.md`	off
`SHOTS_DIR`	`scripts/shots/`	screenshot output folder	`shots-out/screens-1`
`AUDIT_LANGS`	`scripts/audit/`	languages to audit	all five
`SANDBOX_MODELS`, `SANDBOX_LANGS`, `SANDBOX_PROMPTS`	`scripts/sandbox/smoke.config.ts`	bake-off selection	see the config

Variable	Read by	Meaning	Default
`FIT_PORT`, `FIT_DIST`, `FIT_GAMES`, `FIT_PLAY_MS`	game text-fit tours	port, build folder, games, play time	4182, `dist-canvas`

No secrets exist in the repository; `.env`, `.env.local` and `.env.*.local` are git-ignored.

16.3 Release checklist (summary of `docs/RELEASE-CHECKLIST.md`)

A manual list, deliberately outside `verify` (slow and visual):

- `npm run verify` green; screenshot sweep reviewed for horizontal scroll, cut-off text, empty bands and false "Coming soon" promises.
- Initial JS under 200 KB gzip; the test forms (`assessRound` chunk) are in no practice chunk.
- A listening pass (splash, lesson, Fact or Fake stamp then verdict, daily, each game, certificate); test forms stay silent; every sound has an on-screen twin.
- The health reviewer has processed `docs/CONTENT_REVIEW.md` (content stays `unreviewed` until then).
- Partner logos crisp on splash, landing, Home banner and About, at phone and desktop size; creator credit shows on splash, About and footer.
- How-to videos play in all five languages with matching captions; re-record after screen or translation changes.
- User guide refreshed after screen changes (capture, compress, build PDF).
- Print kit rebuilt after content or translation changes (no overflow warnings); Library > Printables lists ten sheets in five languages and `public/print/` contains no test forms.

SECTION 17

Maintenance runbooks

Each runbook ends with the gate: `npm run verify`, plus a one-line entry in `docs/DECISIONS-LOG.md` (date, tag, decision, why). Do not run watch modes or leave servers running.

17.1 Add or edit a Fact or Fake practice item

1. Open `src/content/factOrFake/practice.ts` and copy an item with the same `format`. The next id is `fof-p-NNN` (three digits, unique).
2. Fill every field: `domain`, `truth`, `trick` (`none` for facts, one of the five tricks for fakes), `difficulty` 1 to 3, `module` tag, fictional `author` and `meta`, `text`, `explanation` (why), `how_to_check` (one concrete step) and `basis` (a public source such as a WHO or DOH fact sheet). `review_status` stays `unreviewed` (the file's `base` sets it). No dosing; no real names.
3. Keep each module's slice balanced (fact and fake) and keep the count in the file header in step.
4. Never add `form_A` or `form_B` items here; those live in `assessment.ts` as matched pairs (20 per form, 10 fact and 10 fake, same truth, domain, format, trick and difficulty per `pair_id`).
5. Run `npx vitest run tests/unit/fof-bank.test.ts tests/unit/fof-content.test.ts tests/unit/content-safety.test.ts`.
6. Regenerate the review list: `REVIEW_WRITE=1 npx vitest run tests/unit/contentReview.test.ts`.
7. Translate (17.5) and rebuild the print kit if the item feeds a printed sheet.

17.2 Add a myth

1. Append a call to `myth(id, topic, villain, module, program, { myth, fact, why, check, basis })` in `src/content/library/myths.ts`, inside the right topic comment block. Use a kebab-case unique `id`; `topic` is one of the nine topics, `villain` one of the five tricks, and `program` is `healthph`, `advocaid`, `mosquito`, `aikasalig` or `null`.
2. Respect the limits (myth under 70 characters, fact under 120, "why" at most 40 words and empathetic, "check" at most 25 words), name a public `basis`, use no emoji in health facts, and use person-first language on HIV cards.
3. Optionally link it from a Fact or Fake item with `myth_id` (a test checks that the link resolves).
4. Run `npx vitest run tests/unit/library-content.test.ts`, regenerate the review list, translate, and rebuild the `myth-cards` sheet.

17.3 Change a module

- Lesson text and cards: `src/modules/mNN/index.ts` (typed `ModuleContent`). Keep one micro-interaction per card, `info` text at most about 60 words, and a three-bullet `takeaway`. If the card count changes, update `cards` for that module in `src/content/modules.ts` (it must equal the hook plus the lesson cards).
- Module game: the component in `src/modules/mNN/`; it receives `{moduleId, onComplete(stars)}` and must always be finishable.
- Run `npx vitest run tests/unit/modules-engine.test.ts` and the module's own test file; run the module's e2e spec on all three projects; then translate, regenerate the review list, and refresh the module poster, session guide and lecture video if they changed.

17.4 Add or change an arcade game

Implement a `GameDef` (canvas) or a page component (DOM), add its loader to `src/games/registry.ts`, add an entry with `playable: true` to `src/content/games.ts` (this changes the Daily Challenge rotation modulus, the Home counts and the tutorial list), write a briefing and `howTo`, add pure-logic tests and an e2e spec, and record its how-to video.

17.5 Add a language

1. Add the language to `ALL_LANGUAGES` in `src/app/language.ts` and to `LANGUAGE_NAMES` in `src/sandbox/prompt.ts`; add its words to the red-flag, self-harm, dose and privacy lists in `src/sandbox/guard.ts` and have a native speaker review them.
2. Write `scripts/i18n/glossary/<code>.md` (use `fil.md` as a model) and read `TRANSLATOR.md`.
3. Run the pipeline of section 9.3: extract, batches, translate every `in/` file to `out/`, pack, then `npm run build && npx playwright test tests/e2e/i18n.spec.ts`.
4. Add the code to the language lists in the tooling (`ALL_LANGS` in `scripts/tutorials/lib.mjs` and the print build), then record videos and rebuild the print kit and guide for the new language.
5. Have a native speaker and the health reviewer check the pack before public use.

17.6 Regenerate the PDFs and videos

Deliverable	Commands
User guide	`npm run build`, `npx playwright test -c scripts/guide/capture.config.ts`, `node scripts/guide/compress.mjs`, `node scripts/guide/build-pdf.mjs`
Print kit	`npm run print` (or `node scripts/print/build.mjs --pages`, then inspect `scripts/print/out/png/`)
How-to videos	`npm run build`, `node scripts/tutorials/record.mjs [ids] [--langs=...]`, then rebuild
Lecture videos	`node scripts/lectures/record.mjs mNN --langs=en,fil`
This document	`npm run techdoc`

17.7 Update the Chatbot Sandbox model

1. Pick the candidate (repo, commit hash, file, size, licence) and add or edit its entry in `MODELS` in `src/sandbox/model.ts`; the `revision` must be a full commit hash so the file is pinned. Check its licence.
2. Optionally run the bake-off: `SANDBOX_MODELS=<id> npx playwright test -c scripts/sandbox/smoke.config.ts` (downloads the model).
3. Change `DEFAULT_MODEL_ID`; update `sizeMB` (shown before download and used for the storage check) and any `templateKwargs`.
4. For offline builds: `node scripts/sandbox/fetch-model.mjs <id>` and `VITE_MODEL_SOURCE=local npm run build`.
5. To host the file yourself, copy it to your own Hugging Face repository and change `repo` and `revision`.
6. Re-check the safety lists against sample answers; update `docs/DECISIONS-LOG.md` and the model licence table in section 3.4.

17.8 Change the report relay

Edit `tools/report-relay/Code.gs`, paste it into the Apps Script editor, then choose **Deploy > Manage deployments > Edit > New version** (do not create a new deployment, which changes the URL). Keep `CATEGORIES` in step with `REPORT_CATEGORIES` in `src/app/report.ts`. If the URL changes, update `VITE_REPORT_ENDPOINT` and rebuild.

17.9 Cut a release

1. Update the content review state if reviewers have signed off (`review_status`, `CONTENT_REVIEW.md`).
2. Bump the version in `package.json` if wanted (it feeds the cache name and the in-app version string).
3. Get `npm run verify` green and walk the manual checklist (16.3), including the screenshot sweep.
4. Rebuild the user guide, print kit, videos and this PDF if affected.
5. Commit and push to `main`; the host builds and deploys. Open the live app, reload once so the new service worker activates, and spot-check offline mode and the sandbox.

SECTION 18

Known limitations and roadmap

18.1 Limitations and unverified items

Item	State
Content review	all content ships `unreviewed`; medical and cultural review of `docs/CONTENT_REVIEW.md` is pending
Translations	four packs, machine-produced, 100% of the committed source; native-speaker and health-line review is pending
Live deployment details	Cloudflare account type, build settings and the relay URL in the production build are not recorded in the repository (16.1)
Version labelling	package 0.1.0 and the in-app string "0.1.0 beta" versus "V1.0" on the deliverables
Chatbot quality	a 0.6 billion parameter model gives short and sometimes wrong answers by design (that is what the missions teach); it is not a health source. In the bake-off it answered in 1 to 5 s with good English but mostly echoed questions in Filipino and the regional languages; the app says small AIs are weaker in regional languages
Chatbot hardware	needs WebAssembly SIMD and exception handling, at least 2 GB of memory (when reported) and about 450 MB of free storage; very old phones are routed to Module 3; threads need the COOP and COEP headers (not available on GitHub Pages)
Safari	uses the slower compat build
Model download	378 MB once; depends on Hugging Face availability (mitigation: self-host a copy, 17.7)
Lecture videos	the generator supports all modules, but only Module 1 in English and Filipino was rendered; they are silent (voiceovers planned) and not in the app
Content Security Policy	not set (14.4)
Browser support	modern evergreen browsers (build target `es2022`); installable PWA
Offline	the app works offline after the first visit; videos, print PDFs and the first model download need a connection
Progress portability	progress is per browser and device; there is no sync or export
Test forms in the app	the hidden `#/assess/form_A` and `#/assess/form_B` routes exist for workshops; they are not linked, but a person who knows the URL can open them

18.2 Roadmap (from the decision log and plan documents)

- **Offline apps:** an offline Windows `.exe` and an Android App Bundle with the sandbox model bundled are planned alongside the website; packaging is deferred until the details are final (wrapper, Play Asset Delivery for the model size, webview threading, signing). The switchable model source (`VITE_MODEL_SOURCE=local`) is ready for it (OFFLINE-APPS).
- **Content review:** medical and cultural review, then moving entries from `unreviewed` to `reviewed`; native review of each language and Opus review of health lines.
- **Voiceovers** for the silent module lecture videos, lecture videos for the remaining modules and languages, and the decision whether they join Library > Videos.
- **Further multimedia** from the workplan (module explainer videos), building on the print kit (`docs/PRINT-KIT-PLAN.md`).

- **Facilitator workflow:** workshop pre-test and post-test use of the matched Forms A and B (the paper kit exists; in-app result collection is deliberately not built because there is no backend).
- **Hardening:** a Content Security Policy, a recorded deployment configuration, and a self-hosted copy of the model file.

APPENDIX A

npm scripts and tool commands

Command	Purpose
`npm run dev`	dev server on port 5173
`npm run build`	type-check and build
`npm run preview`	preview on port 4173
`npm run test`	Vitest
`npm run e2e`	Playwright (3 projects)
`npm run budget`	bundle budget check
`npm run verify`	build, budget, test, e2e
`npm run print`	print kit PDFs
`npm run icons`	PWA icons
`npm run techdoc`	this document's PDF
`node scripts/i18n/extract.mjs`	refresh the string source list
`node scripts/i18n/batches.mjs <code>`	create translation batches
`node scripts/i18n/pack.mjs <code>`	build a language pack
`node scripts/i18n/review.mjs <code>`	queue health-line review
`node scripts/tutorials/record.mjs [ids] [--langs=]`	record how-to videos
`node scripts/tutorials/probe.mjs <id>`	plan a game's key timeline
`node scripts/lectures/record.mjs mNN [--langs=]`	record lecture videos
`node scripts/print/build.mjs [--pages] [--only=] [--langs=]`	print kit
`node scripts/guide/compress.mjs`	guide images to WebP
`node scripts/guide/build-pdf.mjs [--pages]`	user guide PDF
`node scripts/sandbox/fetch-model.mjs [id]`	download the model for offline builds
`python scripts/logos/build_logos.py`	build partner logos (needs Pillow and numpy)
`REVIEW_WRITE=1 npx vitest run tests/unit/contentReview.test.ts`	regenerate the content review list

APPENDIX B

Glossary of terms

Term	Meaning
ARAW	AI Relevance Activities and Workshops; "araw" is also Filipino for sun
AIKasalig	the DOST-NICER project on responsible AI in Philippine health care that ARAW belongs to
Ara	the pixel-art sun-spirit mascot and guide
Calibrated trust	trusting the right things for the right reasons, not simply "trusting less"
Villains	the five misinformation tricks: Dr. Fake, Miracle Mike, Fear Monger, Conspiracy Kuya, Kwento Kim
Fact or Fake	the item-judging game; practice items feed play, test Forms A and B feed workshop pre and post tests
Form A, Form B	matched 20-item test sets, hidden from practice, arcade and daily
Truth Radar	two plain meters (discernment and lean) derived from `d'` and `c`
`d'` (d-prime), `c`	signal-detection discernment and response bias, with "fake" as the signal
Daily Challenge	date-seeded game and three Fact or Fake items, the same for everyone on a Manila day
Module game	a module's signature game; never blocks progress
GC	group chat
BHW	Barangay Health Worker
Barangay	the smallest Philippine local government unit
DOH, FDA, DOST, PCHRD, NICER	Department of Health; Food and Drug Administration; Department of Science and Technology; Philippine Council for Health Research and Development; Niche Centers in the Regions for R&D
NCMH	National Center for Mental Health (hotline 1553 in the app)
PWA	progressive web app: an installable, offline-capable web app
OPFS	Origin Private File System: browser storage used for the model file
COOP, COEP	Cross-Origin-Opener-Policy and Cross-Origin-Embedder-Policy headers that enable cross-origin isolation
wllama	WebAssembly build of llama.cpp for the browser (MIT)
GGUF, Q4_K_M	model file format and a 4-bit quantisation scheme
JSPI	JavaScript Promise Integration for WebAssembly
WebVTT, SRT	caption file formats
`review_status`	`unreviewed`, `in_review` or `reviewed` flag on every content entry
Pack	a language file mapping English sentences to translations
Hash router	routing by `#/path` so any static host works
Focus mode	layout without sidebar or tab bar for lessons, games and the certificate

APPENDIX C

Decision-log index

`docs/DECISIONS-LOG.md` holds 239 dated one-line decisions. This index lists one line per tag. The task tags `T00` to `T32` are the build tasks of the first sprint, grouped below; the log has the full text.

Tag	Date	Decision in one line
T00 to T02	2026-10-15	Design tokens and type rules (Press Start 2P for UI, Baloo 2 headings, Nunito 18 px body), stack and versions, splash only on the first load of a session
T03 to T06	2026-10-15	Progress store, starter banks, lesson route in focus mode, Fact or Fake architecture and module tagging
T07 to T12	2026-10-15	Module 1 to 5 lesson structures and review fixes
T13 to T14	2026-10-15	`playable` flag for games; shared engine and GameShell
T15 to T20	2026-10-15	Game rules for Takbo Data!, Feed Mo 'To, Luto ni Ara, Sino 'To? (fictional wallet and bank), Lamok Lamay (two tools) and Tawid Lola!
T21 to T25	2026-10-15 and 16	Hati Tayo!, Gawa Ko 'To!, Tulog na Bea!, Sa Sapatos Nila and OBJECTION!: one game, two faces (module and arcade)
T26 to T32	2026-10-16	Myth of the day uses all myths, Library routes, streak counting, hand-rolled PWA, game data as template data, review file generated from the banks
START-GATE, STARTUP, SPLASH-MOTION	2026-10-16	START waits for the 5-second loading bar; audio attempted on arrival with a first-tap fallback; stepped Ara bob
LANGUAGE-LANDING	2026-10-16	A language landing opens every session; the tap unlocks audio
I18N	2026-10-16	Runtime language packs (English sentence to translation) with English fallback
All-English	2026-10-16	All player text is English in source; only proper names stay Filipino
SOUND	2026-10-16	The approved 8-bit sound set: music per area, effects, Ara's blip voice, switches and volumes
MYTHS-60	2026-10-16	Myths grew from 35 to 61 with named public bases
NICER-MYTHS	2026-10-16	Myths grew from 61 to 88 (outbreaks, HIV, health AI) with an internal program tag
REPORT-RELAY	2026-10-16	Report screen sends through a Google Apps Script web app that emails AIKasalig and logs to a Sheet; Gmail, mailto and copy remain as fallbacks
LAYOUT-FIXES	2026-10-18	A responsive audit (5 languages x 5 viewports) drove CSS fixes
UX-SWEEP	2026-10-20	Screenshot-sweep fixes (Library tabs two by two on phones, stacked partner band)
VIDEO-FINDINGS, GAWA-CONSENT	2026-10-20	Small fixes found while storyboarding videos; the consent footer appears only after a yes
PROJECT-IDENTITY, PARTNER-LOGOS, ABOUT	2026-10-20	ARAW presented as an AIKasalig project: Home banner, six logos in fixed order, About the project
TUTORIAL-VIDEOS, TUTORIAL-PIPELINE	2026-10-20	Silent captioned how-to videos in five languages, recorded from the app with Playwright
SANDBOX	2026-10-21	A real small model runs in the browser through wllama; no server, no API
SANDBOX-HOSTING	2026-10-21	Model file from a pinned Hugging Face commit (approved CDN exception); wllama files served by ARAW
SANDBOX-LOAD	2026-10-21	No scripted fallback; cached in OPFS; unsupported devices get an explanation
SANDBOX-SAFETY	2026-10-21	Red-flag card, self-harm hotline outside the model, privacy nudge, dose filter

Tag	Date	Decision in one line
SANDBOX-FRAMING	2026-10-21	Every answer labelled "can be wrong" with a Usable, Wrong or Get checked rating
SANDBOX-MISSIONS	2026-10-21	Five "Catch the AI" missions at +10 XP, no new badge
SANDBOX-PLACE	2026-10-21	Own route `#/sandbox`, not part of `ARCADE_GAMES`
SANDBOX-HEADERS	2026-10-21	COOP same-origin and COEP credentialless everywhere for threads
SANDBOX-PWA	2026-10-21	Engine files left out of the precache, cached on first use
SANDBOX-MODEL	2026-10-21	Default model Qwen3 0.6B after a 5-model bake-off
SANDBOX-PRELOAD	2026-10-21	No Load button: quiet background download on Wi-Fi or unknown connections
OFFLINE-APPS	2026-10-21	Offline Windows and Android apps planned; packaging deferred
PRINT-KIT	2026-11-03	Printables compiled from typed content by a Vite and Preact print app, printed with Playwright
LECTURES	2026-11-03	Silent slow H.264 MP4 module lectures from the real lesson screen
PRINTABLES-TAB	2026-11-03	Library gets a fifth tab, Printables, downloading on demand
MYTH-CARD-BACKS	2026-11-03	Myth-card backs may go down to about 10.6 pt to fit 8 cards a page
CHECK-BEFORE-SHARE	2026-11-03	The check-before-share sheet stays one page
RED-FLAG-CARD	2026-11-03	Red-flag card adapted from the sandbox lines, no added advice
PRINT-GLOSSARY-SORT	2026-11-03	The printed glossary is sorted by the translated term
TEST-FORMS-PRINT	2026-11-03	Test forms and answer key printed for facilitators only
PRINT-I18N	2026-11-03	Print framing text and lecture captions join the normal translation batches
TECH-DOCS	2026-11-03	This technical documentation and its PDF builder